Unitatea Centrală de Procesare 8086

Student: Apetrei Diana-Andreea

Curs: Structura Sistemelor de Calcul

UNIVERSITATEA TEHNICĂ DIN CLUJ-NAPOCA

ANUL 2025/2026

Contents

1. Introducere ... 6
1.1 Context ... 6
1.2 Obiective ... 6
2. Documentare bibliografică ... 7
2.1 Contextul Istoric și Importanța ... 7
2.2 Arhitectura Internă și Principiul de Funcționare ... 7
2.3 Gestiunea Memoriei: Adresarea Segmentată ... 7
2.4 Moduri de Operare și Interfațare ... 7
3. Analiza ... 9
3.1 Propunerea Proiectului ... 9
3.2 Analiza Arhitecturală ... 9
3.2.1 Analiza Memoriei și a Segmentării ... 9
3.2.2 Analiza Fișierului de Registre (Register File) ... 10
3.2.3 Unitatea Aritmetico-Logică și Flag-urile ... 10
3.3 Analiza Setului de Instrucțiuni și a Controlului ... 10
3.3.1 Simplificarea Ciclului Fetch-Decode ... 10
3.3.2 Subsetul de Instrucțiuni Implementat ... 11
4. Design ... 14
4.1 Arhitectura Generală și Interconectarea Sistemului (test_env) ... 14
4.1.1 Interfața Externă (I/O) ... 14
4.1.2 Logica de Interconectare și Fluxul de Date ... 15
4.2 Unitatea de Interfață cu Magistrala (BIU) ... 15
4.2.1 Interfața Modulului ... 15
4.2.2 Logica Internă și Componentele ... 16
4.2.3 Managementul Instrucțiunilor (Controller BIU) ... 16
4.2.4 Logica de Salt și Flush ... 17
4.3 Registrele de Segment (SegmentsReg) ... 17
4.3.1 Interfața Modulului ... 17
4.4 Calculatorul de Adrese (AddressCalculator) ... 17
4.4.1 Interfața Modulului ... 17
4.4.2 Logica de Calcul ... 18
4.5 Coada FIFO de Instrucțiuni (FIFO) ... 18
4.5.1 Interfața Modulului ... 19
4.6 Unitatea de Execuție (EU) ... 19
4.6.1 Interfața Modulului ... 19

4.6.2 Logica de Multiplexare (Datapath Routing) ... 20
4.7 Unitatea de Control (ControlUnit) ... 21
4.7.1 Interfața Modulului ... 21
4.7.2 Logica Multi-Ciclu (Mașina de Stări Finite) ... 21
4.8 Fișierul de Registre (RegFile) ... 22
4.8.1 Arhitectură și Acces Dual-Port ... 22
4.8.2 Logica de Acces 8-biți vs. 16-biți (Design Conceptual) ... 22
4.8.3 Interfața Modulului ... 23
4.9 Unitatea Aritmetico-Logică (ALU16b) ... 23
4.9.1 Arhitectură și Operații (Design Funcțional) ... 23
4.9.2 Generarea Flag-urilor (Design Dual) ... 23
4.9.3 Interfața Modulului ... 24
4.10 Celula de Calcul pe 1 Bit (OneBitALU) ... 24
4.10.1 Interfața Modulului ... 24
4.11 Registrul de Flag-uri (FlagsRegister) ... 24
4.11.1 Arhitectură și Operații ... 24
4.11.2 Interfața Modulului ... 25
4.12 Unitatea de Memorie Segmentată (Segmented_RAM) ... 25
4.12.1 Structura și Accesul Multi-Port ... 25
4.12.2 Logica de Adresare și Segmentele Alocate ... 25
4.12.3 Interfața Modulului ... 25
4.13 Unitatea de Afișare (SSD) ... 26
4.13.1 Design și Funcționalitate ... 26
4.13.2 Interfața Modulului ... 26
4.14 Generatorul de Puls Unic (MPG) ... 26
4.14.1 Arhitectură și Scop ... 26
4.14.2 Interfața Modulului ... 27
5. Implementare ... 28
5.1 Implementarea Modulului de Top-Level (test_env) ... 28
5.1.1 Logica de Afișare și Debug (SSD Formatting) ... 28
5.2 Implementarea Unității de Interfață cu Magistrala (BIU) ... 28
5.2.1 Logica Controlerului FSM (Procesul BIU_Controller_Process) ... 28
5.2.2 Rutarea Semnalelor de Adresă și Date (VHDL Wiring) ... 29
5.3 Implementarea Registrelor de Segment (SegmentsReg) ... 29
5.3.1 Logica de Inițializare și Actualizare IP ... 29
5.3.2 Constantele de Inițializare ... 30

5.4 Implementarea Calculatorului de Adrese (AddressCalculator) 30
5.4.1 Logica de Calcul a Adresei Fizice (Segmentare) 30
5.4.2 Logica de Control a Contorului de Instrucțiuni (IP) 31
5.5 Implementarea Cozii FIFO de Instrucțiuni (CFIFO) 31
5.5.1 Logica Sincronă a Pointerilor 31
5.5.2 Logica de Prioritate (Flush) 32
5.5.3 Ieșiri Combinatorii de Stare 32
5.6 Implementarea Unității de Execuție (EU) 32
5.6.1 Logica de Selecție a Operanzilor ALU (Multiplexare) 32
5.6.2 Logica de Extensie a Valorilor Imediate 33
5.6.3 Selecția Flag-urilor Corecte 33
5.7 Implementarea Unității de Control (ControlUnit) 33
5.7.1 Decodificarea Instrucțiunilor și Extracția Câmpurilor 33
5.7.2 Logica FSM și Microcodul Secvențial 34
5.8 Implementarea Fișierului de Registre (RegFile) 35
5.8.1 Logica de Scriere Selectivă (Process Write_Process) 35
5.8.2 Implementarea Căilor de Citire Combinatorii 36
5.9 Implementarea Unității Aritmetico-Logice (ALU16b) 36
5.9.1 Construcția Nucleului Aritmetic (Structural VHDL) 36
5.9.2 Logica de Selecție și Deplasare 36
5.9.3 Calculul Dual al Flag-urilor (Logica Combinatorie) 37
5.10 Implementarea Celulei de Calcul pe 1 Bit (OneBitALU) 37
5.10.1 Logica Aritmetică de Bază (Full Adder/Subtractor) 37
5.10.2 Multiplexorul de Funcție (Logica de Selecție) 38
5.11 Implementarea Registrului de Flag-uri (FlagsRegister) 38
5.11.1 Logica de Actualizare Sincronă 38
5.12 Implementarea Unității de Memorie Segmentată (Segmented_RAM) 38
5.12.1 Inițializarea Programului (Funcția Impură InitRam) 39
5.12.2 Logica de Conversie a Adresei (Address Decoder) 39
5.12.3 Procesul de Acces Multi-Port 39
5.13 Implementarea Unității de Afișare (SSD) 39
5.13.1 Logica de Scanare și Contorul Sincron 39
5.13.2 Logica de Decodificare Hexa-7 Segmente 40
5.14 Implementarea Generatorului de Puls Unic (MPG) 40
5.14.1 Logica de Filtrare (Debouncing) 40
5.14.2 Generarea Pulsului Unic (Mono Pulse) 41

6. Testare și Validare 42
6.1 Metodologia de Testare (Single Stepping) 42
6.1.1 Configurația de Test Hardware 42
6.2 Scenariul de Testare (Programul de Referință) 42
6.2.1 Structura Programului 42
6.2.2 Secvența de Execuție și Rezultate 43
6.3 Analiza Rezultatelor 44
7. Concluzii 45
7.1 Obiective Atinse și Contribuții 45
7.2 Validare și Rezultate 45
7.3 Perspective de Viitor 45
8. Bibliografie 46

1. Introducere

1.1 Context

Scopul acestui proiect este proiectarea și implementarea unui model simplificat al unității centrale de prelucrare (CPU) Intel 8086, utilizând limbajul de descriere hardware VHDL. Microprocesorul Intel 8086 reprezintă una dintre cele mai influente arhitecturi din istoria calculatoarelor, stând la baza familiei de procesoare x86 folosite și astăzi în sistemele moderne. Arhitectura sa introduce o structură internă pe 16 biți, un model de adresare segmentată a memoriei și un set flexibil de instrucțiuni care permite efectuarea de operații aritmetice și logice.

Reproiectarea unui model simplificat al acestei arhitecturi oferă o înțelegere practică și educațională a modului în care procesoarele timpurii funcționau la nivel hardware. Proiectul se axează pe implementarea procesorului 8086 folosind componente digitale de bază - bistabile (flip-flop-uri) pentru registre și porți logice combinaționale pentru realizarea operațiilor aritmetice și logice din cadrul unității aritmetico-logice (ALU). Această abordare de tip *bottom-up* (de la nivelul hardware elementar spre sistemul complet) asigură o înțelegere clară a modului în care instrucțiunile sunt executate și a felului în care datele sunt transferate între registre și unități funcționale.

Spre deosebire de simulările software, această implementare își propune să reproducă comportamentul hardware esențial al procesorului 8086, evidențiind componentele sale principale: fișierul de registre, unitatea aritmetico-logică (ALU) și logica de control. Procesorul suportă aproximativ douăzeci de instrucțiuni, acoperind categoriile aritmetice, logice, de transfer de date și de control (salturi). Acestea permit execuția de programe simple și ilustrează conceptele fundamentale ale arhitecturii procesoarelor, precum fluxul de date, modurile de adresare și secvențierea instrucțiunilor.

1.2 Obiective

Obiectivul principal al acestui proiect este construirea unei arhitecturi funcționale de CPU inspirate din modelul Intel 8086, complet descrisă în VHDL și sintetizabilă pe o platformă FPGA utilizând mediul Xilinx Vivado. Designul va fi modular și scalabil, pentru a permite simularea, testarea și înțelegerea individuală a fiecărui bloc funcțional.

Procesorul va include:

- Un fișier de registre complet, alcătuit din opt registre generale (AX - acumulator dedicat, BX, CX, DX - fiecare împărțit în părțile High și Low);
- Suport pentru trei moduri de adresare: imediată, directă și prin registru dedicat;
- O unitate aritmetico-logică (ALU) construită integral din porți logice și bistabile, capabilă să efectueze operații aritmetice și logice;
- O unitate de control, responsabilă de decodificarea instrucțiunilor și coordonarea fluxului de date între registre și ALU;
- Implementarea unui set limitat de instrucțiuni (aproximativ 20) reprezentând principalele categorii ale setului de instrucțiuni 8086.

De asemenea, proiectul își propune validarea funcționalității design-ului prin implementare directă pe o platformă FPGA. Verificarea corectitudinii execuției instrucțiunilor și a actualizării registrelor se va realiza cu ajutorul componentelor de intrare/ieșire disponibile, precum LED-uri, switch-uri sau afișaje cu șapte segmente. Astfel, procesorul poate fi programat și testat în timp real pe hardware, în scop demonstrativ.

2. Documentare bibliografică

2.1 Contextul Istoric și Importanța

Microprocesorul Intel 8086, lansat în anul 1978 de compania Intel, reprezintă un moment de referință în evoluția arhitecturilor de calcul. Acesta a introdus pentru prima dată o arhitectură pe 16 biți, fiind precursorul familiei x86, care domină până astăzi piața procesoarelor pentru calculatoare personale și sisteme embedded.

Dezvoltarea sa a fost motivată de necesitatea unui procesor mai performant decât generațiile anterioare (Intel 8080 și 8085), capabil să suporte un spațiu de memorie mai mare și un set extins de instrucțiuni, păstrând în același timp compatibilitatea cu ecosistemul existent. În timp ce 8080 și 8085 aveau o magistrală de date pe 8 biți și un spațiu de adresare de 64 KB, 8086 a trecut la o arhitectură de date pe 16 biți și o magistrală de adrese de 20 de biți, permițând accesul la 1 MB de memorie.

Prin combinarea unor concepte inovatoare - cum ar fi adresarea segmentată, unitatea de execuție separată de unitatea de interfață cu magistrala (EU/BIU) și prefetch-ul de instrucțiuni - 8086 a devenit modelul de bază pentru procesoarele ulterioare din familia x86, precum 80286, 80386 și Pentium.

2.2 Arhitectura Internă și Principiul de Funcționare

Arhitectura internă a lui 8086 este fundamentală pentru înțelegerea microprocesoarelor moderne. Conform literaturii de specialitate (Brey, 2002; Baruch, 2000), 8086 este împărțit în două unități funcționale principale care lucrează în paralel (conceptul de *pipeline* pe două nivele):

1. **Unitatea de Interfață cu Magistrala (Bus Interface Unit - BIU):** Responsabilă cu preluarea (fetch) instrucțiunilor din memorie și gestiunea tuturor transferurilor de date și adrese pe magistralele externe. BIU conține registrele segment (CS, DS, SS, ES) și coada de pre-fetch (pe 6 octeți), care permite aducerea proactivă a următoarelor instrucțiuni, sporind astfel viteza de execuție.

2. **Unitatea de Execuție (Execution Unit - EU):** Responsabilă cu decodarea și execuția efectivă a instrucțiunilor. EU conține Unitatea Aritmetică și Logică (UAL), setul de registre generale pe 16 biți (AX, BX, CX, DX, SI, DI, SP, BP) și Registrul de Flaguri (Flags).

2.3 Gestiunea Memoriei: Adresarea Segmentată

Un aspect cheie al arhitecturii 8086, care a extins capacitatea de adresare, este schema de adresare segmentată. Deși dispune de registre interne de 16 biți, 8086 poate adresa MegaOctet (MB) de memorie utilizând o magistrală de adrese de 20 de biți (Intel, 1978).

Adresa fizică se calculează prin combinarea conținutului unui registru segment (ex: CS, DS) cu un decalaj (offset), ambele pe 16 biți, conform formulei:

$$\textit{Adresa Fizică} = \textit{Segment} \times 16 + \textit{Offset}$$

Acest mecanism a permis organizarea memoriei în segmente de 64KB și a fost esențial pentru gestionarea resurselor în sistemele de operare timpurii.

2.4 Moduri de Operare și Interfațare

8086 poate funcționa în două moduri, determinate de pinul MN/$\overline{\text{MX}}$:

- Modul Minim: Potrivit pentru sisteme simple, monoprocesor. 8086 generează direct toate semnalele de control necesare.

- Modul Maxim: Proiectat pentru sisteme multiprocesor complexe, unde controlul magistralelor este cedat către un controler de magistrală extern (ex. 8288), permițând utilizarea unor coprocesoare dedicate (ex. 8087 pentru operații în virgulă mobilă) sau a tehnologiei DMA (Direct Memory Access) .

3. Analiza

3.1 Propunerea Proiectului

Arhitectura Intel x86 (CISC) este renumită pentru complexitatea sa, caracterizată prin instrucțiuni de lungimi variabile și mecanisme sofisticate de gestiune a memoriei. Dată fiind vastitatea specificațiilor acesteia, o replicare integrală nu este fezabilă în cadrul unui proiect academic. Prin urmare, acest proiect propune dezvoltarea unei versiuni sintetizabile a procesorului 8086, adaptată pentru o execuție eficientă.

Obiectivul principal este proiectarea și validarea unui procesor pe 16 biți care să respecte modelul de programare 8086, dar care să utilizeze o unitate de control bazată pe stări (Multi-Cycle) pentru a simplifica execuția și a elimina necesitatea mecanismelor complexe de predicție sau evitare a hazardelor specifice arhitecturilor pipeline avansate. Această abordare permite menținerea compatibilității logice cu arhitectura de referință, adoptând totodată o filozofie de execuție secvențială și robustă prin standardizarea setului de instrucțiuni.

Obiectivele tehnice și caracteristicile cheie sunt:

- **Calea de date (Data Path) pe 16 biți:** Unitatea aritmetică și magistralele interne vor opera nativ pe cuvinte de 16 biți, replicând funcționalitatea registrelor generale (AX, BX, CX, DX).
- **Set de Instrucțiuni cu Lungime Fixă (16 biți):** O decizie arhitecturală majoră a acestui proiect este constrângerea tuturor instrucțiunilor la o dimensiune fixă de un cuvânt (16 biți). Această simplificare elimină complexitatea decodificării instrucțiunilor de lungimi variabile specifice x86 original, facilitând o execuție rapidă și predictibilă.
- **Adresare Segmentată:** Se păstrează modelul specific Intel de calcul al adresei fizice pe 20 de biți (Segment x 16 + Offset), demonstrând principiul fundamental al gestiunii memoriei în arhitectura 8086.
- **Execuție Secvențială (Multi-Cycle):** Implementarea unei unități de control bazate pe o mașină de stări finite (FSM), care ghidează execuția fiecărei instrucțiuni prin etape distincte (Fetch, Decode, Execute, WriteBack), asigurând stabilitatea sistemului.

3.2 Analiza Arhitecturală

Această secțiune detaliază componentele hardware critice și logica din spatele acestora, adaptată pentru constrângerile unui design pe FPGA.

3.2.1 Analiza Memoriei și a Segmentării

Procesorul 8086 original accesează un spațiu de memorie de 1MB (2^20 octeți) folosind registre de 16 biți. Pentru a emula acest comportament, proiectul implementează o arhitectură de memorie bazată pe segmentare logică.

Analiza sistemului impune utilizarea unei memorii interne unice, dar care este împărțită logic în zone distincte de funcționare. Mecanismul de adresare se bazează pe combinarea valorii dintr-un registru de segment cu un offset, generând astfel o adresă fizică extinsă pe 20 de biți. Această arhitectură permite procesorului să acceseze date și instrucțiuni aflate la adrese superioare limitei de 64KB impuse de un singur registru de 16 biți.

Memoria este organizată utilizând cele patru registre de segment fundamentale ale arhitecturii 8086, fiecare definind punctul de start al unei zone specifice:

1. **Code Segment (CS):** Definește zona de memorie de unde sunt preluate instrucțiunile (adresate prin Instruction Pointer).

2. **Data Segment (DS):** Indică zona implicită pentru variabile și date statice.
3. **Stack Segment (SS):** Gestionează zona rezervată stivei sistemului (LIFO), esențială pentru apelurile de subrutine.
4. **Extra Segment (ES):** Oferă o zonă suplimentară de memorie pentru operații complexe sau manipulare extinsă de date.

Această organizare asigură o separare clară între codul programului și datele manipulate, replicând modelul de execuție protejată și structurată specific familiei x86.

3.2.2 Analiza Fișierului de Registre (Register File)

O provocare specifică arhitecturii 8086, replicată în acest proiect, este eterogenitatea accesului la registre. Designul fișierului de registre este structurat pe două categorii distincte, în funcție de granularitatea accesului:

1. **Registrele Generale (AX, BX, CX, DX):** Acestea necesită o logică de acces duală. Ele pot fi adresate fie ca un cuvânt complet de 16 biți (pentru operații aritmetice standard), fie individual ca octeți (8 biți) pentru părțile High (AH, BH...) și Low (AL, BL...). Această cerință impune implementarea unei logici de scriere selective ("Byte Enable") în unitatea de control, permițând modificarea a doar jumătate din registru fără a altera cealaltă jumătate.
2. **Registrele de Pointeri și Indexare (SP, BP, SI, DI):** Arhitectura include și un set de patru registre auxiliare (Stack Pointer, Base Pointer, Source Index, Destination Index), care sunt adresabile exclusiv pe 16 biți. Deși implementarea curentă a proiectului se concentrează pe funcționalitățile registrelor generale, structura hardware a fost proiectată să includă și aceste registre. Prezența lor latentă oferă o modalitate clară de extindere a proiectului.

3.2.3 Unitatea Aritmetico-Logică și Flag-urile

Controlul fluxului în programele 8086 (instrucțiunile de salt condiționat precum JZ, JC) depinde exclusiv de starea registrului FLAGS. Analiza ALU implică proiectarea acesteia astfel încât, pe lângă rezultatul numeric, să genereze semnale de stare la fiecare operație:

- **Zero Flag (ZF):** Activ dacă rezultatul este 0.
- **Carry Flag (CF):** Activ la depășirea gamei (unsigned).
- **Sign Flag (SF):** Copiază bitul cel mai semnificativ (MSB).
- **Overflow Flag (OF):** Activ la erori de semn (signed).

3.3 Analiza Setului de Instrucțiuni și a Controlului

Standardizarea instrucțiunilor la o lungime fixă de 16 biți transformă arhitectura internă, apropiind eficiența etapei de *Fetch* de cea a procesoarelor RISC, menținând însă semantica CISC a mnemonicelelor.

3.3.1 Simplificarea Ciclului Fetch-Decode

Standardizarea instrucțiunilor la o lungime fixă de 16 biți (față de lungimea variabilă a arhitecturii originale) oferă avantaje majore în etapa de Fetch și Decode:

- **Fetch Determinist:** Citirea instrucțiunii se realizează într-un singur ciclu de ceas, deoarece lățimea magistralei de date (16 biți) este egală cu lungimea instrucțiunii.

- **Decodificare Rapidă:** Opcodul se află întotdeauna la o poziție fixă, simplificând logica decodorului Unitatea de Control, care poate identifica tipul operației prin logică pur combinatorie, fără a parcurge stări de așteptare pentru octeți suplimentari.

Rolul Cozii FIFO

Deși lungimea fixă elimină necesitatea unei cozi de așteptare pentru octeți multipli (așa cum ar fi necesar în arhitectura CISC originală), proiectul păstrează coada FIFO (Componenta CFIFO) pentru a emula principiul de decuplare funcțională al arhitecturii 8086.

- **Decuplarea BIU-EU**: Coada FIFO permite Unității de Interfață cu Magistrala (BIU) să continue să extragă instrucțiuni atomic și constant din memorie (pre-fetch), chiar și atunci când Unitatea de Execuție (EU) este ocupată cu o operație complexă de calcul.
- **Buffer de Stabilitate:** Acest buffer de instrucțiuni asigură că EU nu trebuie să aștepte ciclul de memorie la fiecare pas, contribuind la fluidizarea execuției Multi-Cycle și sporind viteza generală a sistemului.

Prin urmare, FIFO-ul nu este necesar pentru rezolvarea lungimii variabile (ca la 8086 original), ci pentru optimizarea fluxului de date și menținerea fidelității față de arhitectura BIU/EU.

3.3.2 Subsetul de Instrucțiuni Implementat

Proiectul se concentrează pe un subset esențial de instrucțiuni, adaptate formatului compact de 16 biți. Acestea sunt grupate în următoarele categorii funcționale:

- **Transfer de Date:** MOV (între registre), LIR (încărcare imediată - adaptată), LDR/STR (acces memorie).
- **Aritmetice și Logice:** ADD, SUB, AND, OR, XOR, INC, DEC.
- **Controlul Fluxului:** JMP (salt necondiționat), JZ, JNZ, CMP (comparație care actualizează flag-urile).

Formatul instrucțiunilor alese:

Legenda:
OPcode → codul operației
Size → dimensiunea operanzilor (1 pentru 8 biți, 0 pentru 16 biți)
RegDst → registrul destinație
RegRsc → registrul sursă
Addr → adresa
T → tipul operandului 2 (1 pentru adresa din memorie, 0 pentru registru)
Imm → imediat

1. ADD

OPcode(5b) [15-11]	Size(1b) [10]	RegDst(3b) [9-7]	RegSrc/Addr(6b) [6-1]	T(1b) [0]
00000	s	ddd	ssssss	t

2. SUB

OPcode(5b) [15-11]	Size(1b) [10]	RegDst(3b) [9-7]	RegSrc/Addr(6b) [6-1]	T(1b) [0]
00001	s	ddd	ssssss	t

3. ADI

OPcode(5b) [15-11]	Size(1b) [10]	RegDst(3b) [9-7]	Imm(7b) [6-0]
00010	s	ddd	iiiiiii

4. INC

OPcode(5b) [15-11]	Size(1b) [10]	RegDst(3b) [9-7]	Don't care(7b) [6-0]
00011	s	ddd	xxxxxxx

5. DEC

OPcode(5b) [15-11]	Size(1b) [10]	RegDst(3b) [9-7]	Don't care(7b) [6-0]
00100	s	ddd	xxxxxxx

6. CMP

OPcode(5b) [15-11]	Size(1b) [10]	RegDst(3b) [9-7]	RegSrc(3b) [6-4]	Don't care(4b) [3-0]
00101	s	ddd	sss	xxxx

7. MOV

OPcode(5b) [15-11]	Size(1b) [10]	RegDst(3b) [9-7]	RegSrc(3b) [6-4]	Don't care(4b) [3-0]
00110	s	ddd	sss	xxxx

8. LIR

OPcode(5b) [15-11]	Size(1b) [10]	RegDst(3b) [9-7]	Imm(7b) [6-0]
00111	s	ddd	iiiiiii

9. LDR

OPcode(5b) [15-11]	Size(1b) [10]	RegDst(3b) [9-7]	Addr(7b) [6-0]
01000	s	ddd	aaaaaaa

10. STD

OPcode(5b) [15-11]	Size(1b) [10]	RegDst(3b) [9-7]	Addr(7b) [6-0]
01001	s	ddd	aaaaaaa

11. AND

OPcode(5b) [15-11]	Size(1b) [10]	RegDst(3b) [9-7]	RegSrc(3b) [6-4]	Don't care(4b) [3-0]
01010	s	ddd	sss	xxxx

12. OR

OPcode(5b) [15-11]	Size(1b) [10]	RegDst(3b) [9-7]	RegSrc(3b) [6-4]	Don't care(4b) [3-0]
01011	s	ddd	sss	xxxx

13. XOR

OPcode(5b) [15-11]	Size(1b) [10]	RegDst(3b) [9-7]	RegSrc(3b) [6-4]	Don't care(4b) [3-0]
01100	s	ddd	sss	xxxx

14. NOT

OPcode(5b) [15-11]	Size(1b) [10]	RegDst(3b) [9-7]	Don't care(7b) [6-0]
01101	s	ddd	xxxxxxx

15. SHL

OPcode(5b) [15-11]	Size(1b) [10]	RegDst(3b) [9-7]	Imm(5b) [6-2]	Don't care(2b) [1-0]
01110	s	ddd	iiiii	xx

16. SHR

OPcode(5b) [15-11]	Size(1b) [10]	RegDst(3b) [9-7]	Imm(5b) [6-2]	Don't care(2b) [1-0]
01111	s	ddd	iiiii	xx

17. JMP

OPcode(5b) [15-11]	Addr(11b) [10-0]
10000	aaaaaaaaaaa

18. JZ

OPcode(5b) [15-11]	Addr(11b) [10-0]
10001	aaaaaaaaaaa

19. JNZ

OPcode(5b) [15-11]	Addr(11b) [10-0]
10010	aaaaaaaaaaa

20. NOOP

OPcode(5b) [15-11]	Don't care(11b) [10-0]
10011	xxxxxxxxxxx

4. Design

Capitolul de Design detaliază arhitectura hardware a sistemului, care replică structura internă consacrată a procesorului Intel 8086. Designul este modular și ierarhic, fiind organizat în două blocuri funcționale principale care operează asincron una față de cealaltă: Unitatea de Interfață cu Magistrala (BIU) și Unitatea de Execuție (EU). Această decuplare funcțională permite o eficiență sporită, separând logic procesul de aducere a instrucțiunilor (Fetch) de cel de execuție propriu-zisă.

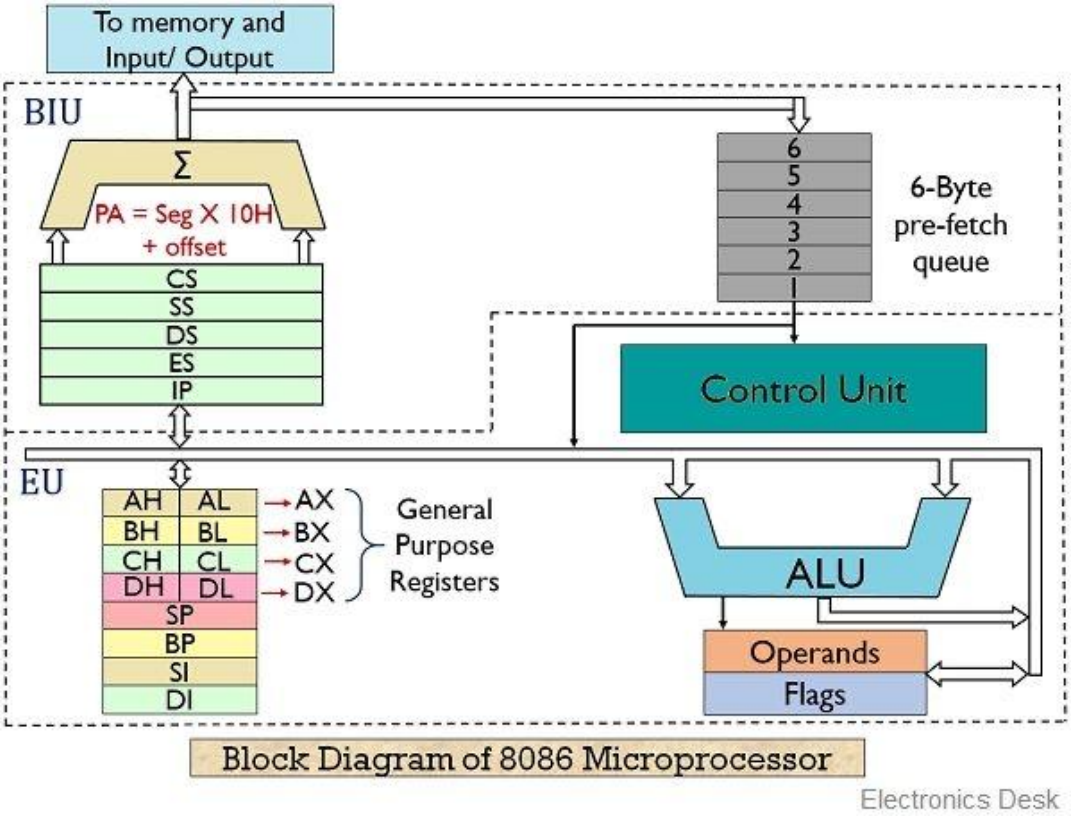

Fig. 1. Diagrama Bloc a Microprocesorului 8086

4.1 Arhitectura Generală și Interconectarea Sistemului (test_env)

Modulul **test_env** servește ca ansamblu de nivel superior (Top-Level) al sistemului, integrând cele două unități centrale (**BIU** și **EU**) cu memoria (**Segmented_RAM**) și perifericele necesare validării pe placa FPGA. Acest modul gestionează, de asemenea, întreaga logică de interfață dintre CPU și utilizator (butoane, afișaj).

4.1.1 Interfața Externă (I/O)

Modulul este proiectat pentru a se mapa direct la pinii plăcii FPGA, simplificând procesul de validare hardware.

Port	Tip	Lățime	Descriere
clk	In	1	Semnalul de ceas principal.
btn	In	5	Butoanele de intrare (utilizate pentru Reset și Single Stepping).
sw	In	16	Switch-uri (utilizate ca selector pentru afișajul SSD).
led	Out	16	LED-uri de ieșire (utilizate pentru afișarea Flag-urilor).
an, cat	Out	8, 7	Ieșiri multiplexate pentru afișajul cu 7 segmente (SSD).

4.1.2 Logica de Interconectare și Fluxul de Date

Fișierul **test_env** stabilește magistralele esențiale și logica de control care leagă **BIU** (Unitatea de Interfață) de **EU** (Unitatea de Execuție) și de memorie (**Segmented_RAM**).

- **Decuplarea BIU-EU:** Modulul **test_env** interconectează **BIU** și **EU** prin multiple magistrale de control și date. **EU** primește instrucțiuni extrase (s_instr_to_eu) și informații despre starea cozii (s_fifo_empty) de la **BIU**. În schimb, **EU** trimite semnale de control (ex: s_fifo_read_en, s_jump_en, s_flush) către **BIU**.
- **Magistralele Memoriei: BIU** generează toate adresele de 20 de biți (s_instr_addr, s_data_addr) și semnalele de scriere (s_ram_write_en, s_ram_load_data), care sunt conectate direct la modulul CRAM (Memoria).
- **Logica de Sincronizare (MPG):** Instanțierea modulului **MPG** (Mono Pulse Generator) este vitală. Acesta filtrează zgomotul (debouncing) de pe butonul de Step (btn(0)), transformând apăsările fizice în semnalul digital curat s_step_pulse, care controlează avansul FSM-ului din **EU**.

4.2 Unitatea de Interfață cu Magistrala (BIU)

Modulul **BIU** (Bus Interface Unit) este responsabil pentru gestionarea tuturor interacțiunilor cu memoria și pentru pre-procesarea instrucțiunilor înainte ca acestea să ajungă la Unitatea de Execuție (**EU**). Logica sa internă replică funcționalitatea procesorului 8086 prin separarea procesului de *fetch* de cel de *execute*, utilizând o mașină de stări simplă pentru a menține coada de instrucțiuni plină.

4.2.1 Interfața Modulului

BIU acționează ca o interfață între CPU (**EU**) și memoria externă.

Categorie	Port	Tip	Lățime	Funcție
Sincronizare	CLK, RST	In	1	Semnale de Ceas și Reset.
I/O Memorie (Adrese)	INSTR_ADDR_OUT	Out	20	Adresa fizică CS:IP pentru extragerea instrucțiunii (Fetch).
	DATA_ADDR_OUT	Out	20	Adresa fizică pentru citirea datelor (DS:Offset).
	RAM_WRITE_ADDR_OUT	Out	20	Adresa fizică pentru scrierea datelor.
I/O Memorie (Date)	INSTR_DATA_IN	In	16	Instrucțiune citită din Memorie.
	DATA_READ_IN	In	16	Date citite din Memorie (pentru operanzi).
	RAM_WRITE_EN_OUT	Out	1	Semnal de activare a scrierii în RAM.
	RAM_LOAD_DATA_OUT	Out	1	Datele ce urmează a fi scrise în RAM.
I/O EU (Control)	READ_EN_FROM_EU	In	1	Cerere de citire instrucțiune nouă din coadă.
	FLUSH_FROM_EU	In	1	Semnal de golire a cozii (la salt).
	JUMP_EN_FROM_EU	In	1	Semnal de salt condiționat sau necondiționat.
	TARGET_ADDR_FROM_EU	In	20	Adresa țintă pentru salt (calculată în EU).

	FIFO_EMPTY_TO_EU	Out	1	Semnal de stare: Coada FIFO este/nu este goală.
I/O EU (Date)	INSTR_OUT_TO_EU	Out	16	Instrucțiunea de la FIFO către EU.
	DATA_OFFSET_FROM_EU	In	16	Offset-ul necesar pentru calculul adresei de date.
	DATA_READ_OUT_TO_EU	Out	16	Date citite din RAM, transmise către EU.
	DATA_WRITE_EN_FROM_EU	In	1	Semnal de activare a scrierii datelor (de la EU).
	DATA_TO_WRITE_FROM_EU	In	16	Datele de scris (sursă: un registru EU).

4.2.2 Logica Internă și Componentele

Modulul **BIU** este construit prin instanțierea a trei sub-module esențiale:

1. **Registrele de Segment (CSegmentsReg):** Stochează valorile registrelor CS, DS, SS, ES (16 biți).
2. **Calculatorul de Adrese (CAddrCalc):** Calculează adresa fizică de 20 de biți pentru instrucțiuni și date.
3. **Coada FIFO (CFIFO):** Buffer pentru instrucțiunile extrase din memorie.

4.2.3 Managementul Instrucțiunilor (Controller BIU)

Logica de bază a **BIU** este gestionată de o Mașină de Stări Finite (FSM) care orchestrează ciclul de *pre-fetch* al instrucțiunilor. FSM-ul acționează independent de **EU** (Unitatea de Execuție), cu scopul de a menține coada **FIFO** plină pentru a asigura continuitatea execuției.

Stare	Descriere conceptuala	Obiectiv Principal
IDLE	Starea de repaus a BIU și punctul de intrare/ieșire al ciclului de fetch. Modulul monitorizează constant semnalele de control primite de la EU și starea de ocupare a Cozii FIFO. Dacă coada indică faptul că nu este plină, BIU inițiază automat secvența de extragere a unei noi instrucțiuni din Memorie.	Monitorizarea stării Cozii FIFO și inițierea extragerii (Fetch).
FETCH_WAIT_1	Această stare este esențială pentru a compensa latența de un ciclu de ceas a Memoriei Segmentate. În timpul acestui ciclu, BIU menține adresa curentă (CS:IP) pe magistrala de adrese. Starea acționează ca un temporizator forțat, asigurând că memoria sincronă are suficient timp pentru a citi adresa și a plasa instrucțiunea pe magistrala de date.	Gestionarea temporizării și a latenței memoriei.
FETCH_WRITE	Starea de finalizare a ciclului de Fetch. În acest moment, datele instrucțiunii citite în ciclul anterior sunt valide și gata de utilizare. BIU execută simultan două acțiuni critice pentru avansarea programului: înregistrarea instrucțiunii în coada FIFO și avansarea Contorului de Instrucțiuni (IP) la următoarea adresă secvențială. Modulul revine apoi imediat la starea IDLE.	Finalizarea tranzacției și actualizarea Contorului de Program (IP).

4.2.4 Logica de Salt și Flush

Logica de control a **BIU** gestionează întreruperea execuției secvențiale:

- **Reset și Flush:** Când semnalul FLUSH_FROM_EU este activat (indicând o decizie de salt reușită), **BIU** resetează FSM-ul la starea IDLE și semnalează modulului CFIFO să se golească.
- **Jump:** Când JUMP_EN_FROM_EU este activ, modulul CAddrCalc actualizează registrul IP la noua adresă țintă furnizată de EU (TARGET_ADDR_FROM_EU), asigurând noul flux al programului.

4.3 Registrele de Segment (SegmentsReg)

Modulul **SegmentsReg** este o componentă sincronă fundamentală în cadrul Unității de Interfață cu Magistrala (**BIU**), având rolul de a stoca valorile pe 16 biți ale registrelor de segment (CS, DS, SS, ES) și ale Contorului de Instrucțiuni (IP). Aceste registre definesc cadrul logic de bază pentru implementarea schemei de adresare segmentată a arhitecturii 8086.

4.3.1 Interfața Modulului

Designul modulului este pur funcțional, acționând ca un stocator pentru valorile necesare calculatorului de adrese. Acesta este sincronizat de ceasul sistemului (CLK) și dispune de un semnal de reset (RST) care inițializează registrele la valorile de pornire.

Singurul registru a cărui valoare se modifică dinamic în timpul ciclului de instrucțiune este Contorul de Instrucțiuni (IP_REG), care este actualizat cu noua adresă de offset furnizată de modulul AddressCalculator prin portul NEW_IP_IN. Celelalte registre de segment rămân constante pe durata execuției, cu excepția cazului în care sunt manipulate direct de instrucțiuni de schimbare a segmentului (care pot fi adăugate în viitor).

Categorie	Port	Tip	Lățime	Descriere
Sincronizare	CLK, RST	In	1	Semnale de Ceas și Reset.
Intrare Date	NEW_IP_IN	In	16	Noua valoare a offset-ului (IP) de la Calculatorul de Adrese.
Ieșire Date	CS_OUT, IP_OUT	Out	16	Registrul Segment Cod și Offset (IP).
	DS_OUT, SS_OUT, ES_OUT	Out	16	Registrele Segment Date, Segment Stivă și Segment Extra.

4.4 Calculatorul de Adrese (AddressCalculator)

Modulul **AddressCalculator** este o componentă pur combinatorie (asincronă) responsabilă pentru conversia adreselor logice (segment:offset) în adrese fizice de 20 de biți, adresabile de memorie.

4.4.1 Interfața Modulului

Modulul primește toate registrele de segment de la **SegmentsReg** și datele de offset de la Unitatea de Execuție (**EU**), generând simultan adresele fizice necesare pentru citirea instrucțiunilor și accesul la date.

Categorie	Port	Tip	Lățime	Descriere
Intrări Registru Segment	CS_IN, IP_IN, DS_IN, SS_IN, ES_IN	In	16	Registrele de segment și Contorul de Instrucțiuni.
Intrare Offset	DATA_OFFSET_IN	In	16	Offset-ul generat de EU pentru accesul la date (Load/Store).

Intrări de Control	JUMP_EN	In	1	Semnal de control al saltului.
	TARGET_ADDR_IN	In	20	Adresa țintă pentru salt (furnizată de EU).
	INCREMENT_IP_EN	In	1	Semnal de avans secvențial al IP (din FSM-ul BIU).
Ieșiri Adresă Fizică	CURRENT_PHYSICAL_ADDR_OUT	Out	20	Adresa fizică CS:IP (pentru Fetch).
	DATA_PHYSICAL_ADDR_OUT	Out	20	Adresa fizică DS:Offset (pentru accesul la date).
	NEXT_IP_OUT	Out	16	Noua valoare a Contorului de Instrucțiuni (IP) după execuție sau salt.

4.4.2 Logica de Calcul

Modulul **AddressCalculator** este un bloc pur combinatoriu a cărui funcție este esențială pentru implementarea schemei de adresare segmentată a procesorului 8086. Procesul intern realizează trei funcții critice în paralel, bazate pe o logică de sumare pe 20 de biți:

1. **Calculul Adresei Fizice Curente (Fetch):**
   - **Funcție:** Modulul combină valoarea registrului de segment (CS_IN) cu offset-ul curent (IP_IN) pentru a genera adresa fizică pe 20 de biți necesară extragerii instrucțiunii din memorie.
   - **Logica:** Segmentul este logic deplasat la stânga cu 4 poziții (echivalentul înmulțirii cu 16), apoi adunat cu offset-ul.
2. **Calculul Adresei Fizice de Date (Load/Store):**
   - **Funcție:** Calculează adresa fizică pentru accesul la operanzi de memorie (Load/Store).
   - **Logica:** Utilizează registrul Data Segment (DS_IN) ca bază și offset-ul furnizat dinamic de Unitatea de Execuție (DATA_OFFSET_IN).
3. **Logica de Avansare și Control a Contorului de Instrucțiuni (IP):**
   - **Funcție:** Modulul determină care va fi următoarea valoare a Contorului de Instrucțiuni (IP), controlând astfel fluxul programului.
   - **Avans Secvențial:** Dacă semnalul de avans secvențial (INCREMENT_IP_EN) este activ, IP este incrementat cu 2 (IP ← IP + 2), reflectând faptul că toate instrucțiunile sunt de lungime fixă (16 biți sau 2 octeți).
   - **Salt (Jump):** Dacă un semnal de salt (JUMP_EN) este primit, modulul ignoră avansul secvențial și selectează adresa țintă furnizată de Unitatea de Execuție. Această logică de selecție asigură tranziția corectă a programului la adrese noi, fie ele relative sau absolute.

4.5 Coada FIFO de Instrucțiuni (FIFO)

Modulul **CFIFO** (Coada de tip First-In-First-Out) este o componentă sincronă esențială în cadrul **BIU**, având rolul de a simula mecanismul de pre-fetch al procesorului 8086. Aceasta acționează ca un buffer între ciclul de aducere a instrucțiunilor din memorie (Fetch) și ciclul de execuție al acestora

(Decode/Execute), asigurând decuplarea funcțională a celor două unități. Coada este dimensionată la o adâncime fixă de 3 cuvinte (3 x 16 biți).

4.5.1 Interfața Modulului

Logica internă utilizează pointeri sincroni (Q_HEAD pentru citire și Q_TAIL pentru scriere) și un contor de ocupare (Q_COUNT). Modulul furnizează semnale de stare critice către Unitatea de Control a **BIU** și către **EU**, influențând direct fluxul de execuție al programului.

Categorie	Port	Tip	Lățime	Descriere
Sincronizare	CLK, RST	In	1	Semnale de Ceas și Reset.
Control Acces	WRITE_EN	In	1	Semnal de Scriere: BIU a adus o nouă instrucțiune.
	READ_EN	In	1	Semnal de Citire: EU cere următoarea instrucțiune (consumă un cuvânt).
	FLUSH	In	1	Semnal de Golire: Activ la un salt necondiționat (pentru a invalida instrucțiunile pre-extrase).
Intrare/Ieșire Date	INSTRUCTION_IN	In	16	Instrucțiunea citită din Memorie (date de intrare)
	INSTRUCTION_OUT	Out	16	Instrucțiunea de la capul cozii către Unitatea de Execuție (EU).
Ieșiri de Stare	FULL	Out	1	Semnal de Stare: Coada este plină (BIU trebuie să aștepte).
	EMPTY	Out	1	Semnal de Stare: Coada este goală (EU trebuie să aștepte).
Ieșiri de Debug	Q_OUT_0, Q_OUT_1, Q_OUT_2	Out	16	Conținutul cozii pentru monitorizare.

Obiectiv Funcțional:

- **Citirea:** Este activată de READ_EN (cererea **EU**). Dacă coada nu este goală, pointerul Q_HEAD avansează, iar instrucțiunea este furnizată pe INSTRUCTION_OUT.
- **Prioritate (Flush):** Semnalul FLUSH are prioritate maximă, resetând instantaneu pointerii și contorul, eliminând conținutul invalid după un salt.

4.6 Unitatea de Execuție (EU)

Modulul **EU** (Execution Unit) este nucleul de calcul al procesorului. Acesta primește instrucțiunile pre-extrase de la **BIU** și le execută prin coordonarea operațiilor în Registre, ALU și Registrul de Flag-uri. **EU** acționează ca un Datapath centralizat, conținând logica de multiplexare și cablajul esențial care permite transferul datelor între componente.

4.6.1 Interfața Modulului

Modulul EU este o entitate complexă care face legătura între periferice (Buton Step), BIU (FIFO, control salt) și componentele interne.

Categorie	Port	Tip	Lățime	Descriere
Intrări de Sincronizare	CLK, RST	In	1	Semnale de Ceas și Reset.
	STEP_BUTTON_IN	In	1	Semnal de avans pas-cu-pas (din MPG).

Intrări Date & Control	INSTRUCTION_IN	In	16	Instrucțiunea de la FIFO (BIU).
	FIFO_EMPTY	In	1	Starea cozii (pentru a bloca execuția la coadă goală).
	MEM_DATA_IN	In	16	Date citite din Memorie (pentru operații Load).
Ieșiri de Control (către BIU)	FIFO_READ_EN	Out	1	Semnal de citire/consum instrucțiune din coada FIFO.
	BIU_FLUSH	Out	1	Semnal de golire imediată a cozii (la salt necondiționat).
	BIU_JUMP_EN	Out	1	Semnal de activare a saltului.
	BIU_JUMP_TARGET	Out	20	Adresa țintă (CS:IP) pentru salt.
	DATA_OFFSET_OUT	Out	16	Offset-ul adresei pentru operații Load/Store.
Ieșiri de Scriere Memorie	MEM_WRITE_EN	Out	1	Semnal de activare a scrierii în Memorie.
	MEM_DATA_OUT	Out	16	Datele de scris în Memorie (Sursă: Registru EU).
Ieșiri de Monitorizare & Stare	AX_OUT, BX_OUT, CX_OUT, DX_OUT	Out	16	Valorile registrelor generale (pentru afișare/debug).
	CURRENT_FLAGS_OUT	Out	4	Starea curentă a Flag-urilor (ZF, CF, SF, OF).
	IR_OUT	Out	16	Registrul de Instrucțiuni (pentru afișarea instrucțiunii în execuție).

4.6.2 Logica de Multiplexare (Datapath Routing)

Logica internă a modulului EU este definită de multiplexoarele care direcționează datele de la sursa corectă către ALU, sub comanda Unității de Control.

1. **Intrarea A a ALU:** Această intrare este alimentată de Portul A al Registrului și include logica de adaptare a lățimii datelor. Dacă operația este pe 8 biți (ex: ADD AL, BH), datele de 8 biți sunt extinse la 16 biți pentru a se potrivi cu lățimea completă a ALU.

2. **Intrarea B a ALU:** Această intrare este controlată de un multiplexor complex care selectează sursa celui de-al doilea operand:

    - **Registru:** Operandul B este preluat de la Portul B al Registrului.
    - **Memorie:** Operandul B este preluat din memoria de date (MEM_DATA_IN).
    - **Imediat:** Operandul B este o constantă imediată extrasă și extinsă din instrucțiunea curentă.

3. **Sursa Finală a Flag-urilor:** Există o logică de selecție finală care alege setul corect de flag-uri de la ALU (calculat pentru 8 biți sau pentru 16 biți) înainte de a fi stocat în Registrul de Flag-uri.

4.7 Unitatea de Control (ControlUnit)

Modulul **ControlUnit** este centrul decizional al Unității de Execuție (**EU**) și, implicit, al întregului procesor multi-ciclu. Implementat ca o Mașină de Stări Finite (FSM), rolul său este de a interpreta instrucțiunile și de a genera toate semnalele de control (timp, adresare, operații ALU) necesare pentru a dirija fluxul de date corect în Datapath.

4.7.1 Interfața Modulului

Unitatea de Control acționează ca un distribuitor de semnale, primind date de stare și control din exterior și emițând comenzi către toate sub-modulele EU și BIU.

Categorie	Port	Tip	Lățime	Descriere
Intrări de Sincronizare	CLK, RST	In	1	Semnale de Ceas și Reset.
	STEP_BUTTON_IN	In	1	Semnal de avans pas-cu-pas (din MPG).
Intrări de Stare & Date	INSTRUCTION_IN	In	16	Instrucțiunea decodificată (Opcode + Operanzi).
	MEM_DATA_IN	In	16	Date citite din Memorie (pentru operații Load).
	CURRENT_FLAGS	In	4	Starea curentă a Flag-urilor (ZF, CF, SF, OF).
	FIFO_EMPTY	In	1	Starea cozii (pentru a bloca execuția).
Ieșiri de Adresare & Salt	BIU_JUMP_TARGET	Out	20	Adresa țintă (CS:IP) pentru salt.
	DATA_OFFSET_OUT	Out	16	Offset-ul necesar pentru calculul adresei de date.
	REG_READ_ADDR_A, REG_READ_ADDR_B	Out	3	Adrese pentru citirea operanzilor sursă/operand B.
	REG_WRITE_ADDR	Out	3	Adresa registrului destinație.
Ieșiri de Control (Comenzi)	ALU_OP_SEL	Out	4	Codul operației aritmetice/logice pentru ALU.
	ALU_IN_B_MUX_SEL	Out	2	Selector pentru sursa Operandului B (Regs/Mem/Imm).
	SIZE_SEL_8_16	Out	1	Selector lățime date (8 biți vs. 16 biți).
	REG_WRITE_EN	Out	1	Activare scriere în Registrul General.
	FLAG_REG_WRITE_EN	Out	1	Activare scriere în Registrul de Flag-uri.
	MEM_WRITE_EN	Out	1	Activare scriere în Memoria RAM.
	FIFO_READ_EN	Out	1	Cerere de consum al instrucțiunii din FIFO.
	BIU_FLUSH, BIU_JUMP_EN	Out	1	Semnale de activare a golirii cozii și a saltului.
Ieșiri de Debug	IR_OUT	Out	16	Registrul de Instrucțiuni (pentru afișare/debug).

4.7.2 Logica Multi-Ciclu (Mașina de Stări Finite)

FSM-ul este proiectat cu șase stări principale care ghidează execuția instrucțiunilor. Decizia de a folosi șase stări asigură tratarea corectă a latenței memoriei și a operațiilor complexe (Load/Store).

Stare	Rol Funcțional Principal	Instrucțiuni Aferente
s_IDLE	Stare de așteptare. Procesorul așteaptă semnalul STEP_BUTTON_IN și confirmarea că FIFO nu este gol pentru a inița un nou ciclu de execuție.	Toate instrucțiunile.
s_DECODE	Decodifică opcodul din instrucțiune. În această stare sunt setate toți selectorii MUX și codurile de operație ALU (ALU_OP_SEL) pentru ciclul următor. Decizia de tranziție se ia aici (EXECUTE, MEM_READ, sau JUMP).	Toate instrucțiunile.
s_MEM_READ	Stare dedicată gestionării latenței necesare pentru citirea datelor din Memoria RAM pentru operanzii de memorie. FSM-ul menține comanda de citire activă și pregătește Datapath-ul pentru ciclul de execuție.	LDR, ADD/SUB (cu operand memorie).
s_EXECUTE	Ciclu unic de ceas în care se finalizează operația ALU. Aici se generează Flag-urile și se activează semnalul REG_WRITE_EN pentru a scrie rezultatul înapoi în fișierul de registre.	Operații aritmetice, logice și de transfer.
s_MEM_WRITE	Stare dedicată operației de scriere în memorie (ex: STD). Unitatea de Control activează semnalul MEM_WRITE_EN și furnizează adresa necesară.	STD.
s_JUMP	Stare critică dedicată evaluării condițiilor de salt. Modulul citește Flag-urile (CURRENT_FLAGS) și, dacă condiția este îndeplinită (ex: JZ și ZF=1), emite semnalele BIU_JUMP_EN și BIU_FLUSH.	JMP, JZ, JNZ.

4.8 Fișierul de Registre (RegFile)

Modulul **RegFile** implementează cele opt registre de 16 biți ale arhitecturii 8086, servind drept stocare principală pentru operanzii ALU. Designul său complex este adaptat specific pentru a permite accesul dual pe 8 și 16 biți, o cerință esențială a setului de instrucțiuni x86.

4.8.1 Arhitectură și Acces Dual-Port

Fișierul de registre este proiectat ca o componentă cu o singură poartă de scriere (sincronă) și două porți de citire (combinatorii).

- **Implementare Internă:** Modulul stochează opt registre pe 16 biți, incluzând registrele generale (AX, BX, CX, DX) și registrele de pointeri și indexare (SP, BP, SI, DI).
- **Porturi de Citire (A și B):** Cele două porți de citire asincrone sunt vitale pentru calea de date, deoarece permit ALU să preia ambii operanzi necesari unei operații într-un singur ciclu de ceas. Fiecare port de citire furnizează simultan datele pe 16 biți (DATA_RD_A_16) și pe 8 biți (DATA_RD_A_8).

4.8.2 Logica de Acces 8-biți vs. 16-biți (Design Conceptual)

Designul modulului gestionează complexitatea registrelor generale (AX, BX, CX, DX) care pot fi accesate ca sub-registre de 8 biți (AH/AL).

- **Scriere Selectivă (Write Process):** Logica de scriere este controlată de semnalul SIZE_SEL (0=Word, 1=Byte) și de adresa de 3 biți (ADDR_WR). Când operația este pe 8 biți, logica internă folosește structura adresei pentru a distinge între octetul High (AH, BH...) și octetul Low

(AL, BL...), realizând o actualizare parțială condițională care scrie doar în porțiunea de 8 biți vizată, lăsând restul registrului intact.

- **Registrele de Pointeri:** Registrele SP, BP, SI, DI sunt tratate ca fiind adresabile doar pe 16 biți, deoarece arhitectura 8086 nu permite accesul pe octeți la aceste registre.

4.8.3 Interfața Modulului

Categorie	Port	Tip	Lățime	Descriere
Sincronizare	CLK, RST	In	1	Semnale de Ceas și Reset.
Control Scriere	SIZE_SEL	In	1	Selector lățime (0=16 biți, 1=8 biți).
	WR_EN	In	1	Activare scriere.
	ADDR_WR	In	3	Adresa registrului țintă (000-111).
	DATA_WR	In	16	Datele de scris (rezultatul ALU).
Ieșiri Citire Port A	ADDR_RD_A	In	3	Adresa registrului A.
	DATA_RD_A_16	Out	16	Ieșire 16 biți (Word).
	DATA_RD_A_8	Out	8	Ieșire 8 biți (Byte).
Ieșiri Citire Port B	ADDR_RD_B	In	3	Adresa registrului B.
	DATA_RD_B_16	Out	16	Ieșire 16 biți (Word).
	DATA_RD_B_8	Out	8	Ieșire 8 biți (Byte).
Ieșiri Monitorizare	AX_OUT, BX_OUT, CX_OUT, DX_OUT	Out	16	Ieșiri dedicate pentru display (debug).

4.9 Unitatea Aritmetico-Logică (ALU16b)

Modulul **ALU16b** este o componentă pur combinatorie care primește operanzi de la Registre și coduri de operație de la Unitatea de Control, având rolul de a executa toate calculele și de a determina starea procesorului (Flag-uri).

4.9.1 Arhitectură și Operații (Design Funcțional)

Designul ALU este definit prin existența a trei canale de procesare care funcționează în paralel:

1. **Nucleul Aritmetic:** Responsabil de operațiile de adunare, scădere, incrementare și decrementare. Logica sa trebuie să gestioneze propagarea bitului de transport (Carry).
2. **Blocul Logic:** Implementează operațiile booleene (AND, OR, XOR, NOT).
3. **Blocul de Shiftare:** Implementează deplasările pe biți (SHL, SHR), bazându-se pe valoarea imediată de deplasare (SHIFT_IMM).

Rezultatul final (ALU_OUT) este selectat printr-un multiplexor mare, în funcție de codul de operație primit (ALU_OP_SEL).

4.9.2 Generarea Flag-urilor (Design Dual)

O cerință esențială pentru compatibilitatea cu x86 este calcularea simultană a Flag-urilor pentru lățimi diferite de operanzi. Modulul ALU16b calculează două seturi complete de Flag-uri în paralel:

- **Setul de 16 biți:** Bazat pe rezultatul complet (Word), necesar pentru operațiile pe registre întregi.
- **Setul de 8 biți:** Bazat doar pe cei 8 biți inferiori (Low Byte) ai rezultatului, necesar pentru operațiile pe octeți (ex: ADD AL, BL).

4.9.3 Interfața Modulului

Categorie	Port	Tip	Lățime	Descriere
Control	ALU_OP_SEL	In	4	Codul operației selectate (ADD, SUB, etc.).
Intrări Date	DATA_IN_A, DATA_IN_B	In	16	Operanzii A și B (de obicei din RegFile).
	SHIFT_IMM	In	5	Cantitatea de deplasare pentru operațiile SHL/SHR.
Ieșiri Rezultat	ALU_OUT	Out	16	Rezultatul final al operației (care se scrie în registru).
Ieșiri Flag-uri	FLAGS_OUT_16	Out	4	Flag-uri de stare (S, O, C, Z) pentru operații pe 16 biți.
	FLAGS_OUT_8	Out	4	Flag-uri de stare (S, O, C, Z) pentru operații pe 8 biți.

4.10 Celula de Calcul pe 1 Bit (OneBitALU)

Modulul **OneBitALU** este celula logică fundamentală, pur combinatorie, care este instanțiată de 16 ori (prin generate statement) pentru a forma Nucleul Aritmetic al ALU-ului pe 16 biți. Rolul său este de a executa o funcție selectată (Aritmetică, AND, OR, XOR) la nivel de bit și de a propaga bitul de transport/împrumut (C_OUT) de-a lungul lanțului.

4.10.1 Interfața Modulului

Categorie	Port	Tip	Lățime	Descriere
Intrări Date	A_IN, B_IN	In	1	Operanzi de intrare pe un singur bit.
Control	FUNC_SEL	In	2	Selector de funcție (Aritmetică, AND, OR, XOR).
	SUB_SEL	In	1	Selector Scădere (1 = Scădere, inversează B_IN).
	C_IN	In	1	Bitul de Transport/Împrumut de la celula precedentă.
Ieșiri	RESULT	Out	1	Bitul de rezultat (ieșirea funcției selectate).
	C_OUT	Out	1	Bitul de Transport/Împrumut către celula următoare.

4.11 Registrul de Flag-uri (FlagsRegister)

Modulul FlagsRegister este un registru sincron de 4 biți care stochează flag-urile de stare generate de ALU. Aceste flag-uri (Sign, Overflow, Carry, Zero) sunt critice pentru instrucțiunile de control al fluxului (ex: JZ, JNZ), deoarece Unitatea de Control citește valoarea lor pentru a determina dacă un salt condiționat trebuie executat.

4.11.1 Arhitectură și Operații

Registrul este implementat ca un modul sincron (gestionat de CLK) cu o singură poartă de scriere controlată de semnalul FLAG_WRITE_EN.

- **Flag-uri Stocate:** Modulul stochează patru flag-uri esențiale (3 downto 0): S (Sign), O (Overflow), C (Carry), și Z (Zero).
- **Actualizare Sincronă:** Actualizarea valorii Flag-urilor se realizează exclusiv la frontul crescător al ceasului și numai când semnalul FLAG_WRITE_EN este activ (de exemplu, după o operație aritmetică sau de comparație, dar nu după un MOV sau LDR).
- **Reset:** La semnalul de Reset (RST), toate flag-urile sunt inițializate la valoarea 0000.

4.11.2 Interfața Modulului

Categorie	Port	Tip	Lățime	Descriere
Sincronizare	CLK, RST	In	1	Semnale de Ceas și Reset.
Intrare Scriere	FLAG_WRITE_EN	In	1	Activare scriere (de la Unitatea de Control).
	FLAGS_IN	In	4	Noua valoare a Flag-urilor (din ALU).
Ieșire Citire	FLAGS_OUT	Out	4	Starea curentă a Flag-urilor (citită de Unitatea de Control).

4.12 Unitatea de Memorie Segmentată (Segmented_RAM)

Modulul Segmented_RAM simulează spațiul de memorie adresabil al procesorului. Deși arhitectura 8086 adresează 1MB, modulul alocă o porțiune gestionabilă de 514 cuvinte (16 biți) din memoria internă a FPGA-ului. Această memorie este organizată în segmente logice.

4.12.1 Structura și Accesul Multi-Port

Designul modulului permite accesul dual simultan la memorie, optimizat pentru arhitectura **BIU-EU**:

1. **Port Citire Instrucțiuni (INSTR_OUT):** Acest port este sincron (latch-uit), utilizat exclusiv de BIU în etapa de Fetch.
2. **Port Citire Date (DATA_READ):** Acest port este combinatoriu (asincron), permițând accesul rapid al EU la operanzi în orice moment.
3. **Port Scriere Date:** Acest port este sincron, activat de semnalul WRITE_EN pentru operațiile de tip Store.

4.12.2 Logica de Adresare și Segmentele Alocate

Modulul gestionează conversia adreselor: primește adrese fizice de 20 de biți de la AddressCalculator și le convertește în indexul matricial intern.

Spațiul de 514 cuvinte este alocat static, respectând segmentele logice definite în faza de analiză:

Segment	Cuvinte Alocate	Scop
Code Segment	256	Stocarea programului (instrucțiunilor).
Data Segment	256	Variabile și operanzi.
Stack Segment	1	Stiva (pentru extinderi viitoare).
Extra Segment	1	Stocare auxiliară (pentru extinderi viitoare).

4.12.3 Interfața Modulului

Categorie	Port	Tip	Lățime	Descriere
Sincronizare	CLK, RST	In	1	Semnale de Ceas și Reset.
Port Instrucțiuni	INSTR_ADDR	In	20	Adresa fizică CS:IP (Fetch).
	INSTR_OUT	Out	16	Instrucțiunea (latch-uită, sincronă).
Port Citire Date	DATA_READ_ADDR	In	20	Adresa fizică pentru citirea operanzilor.
	DATA_READ	Out	16	Datele citite (combinatoriu/asincron).
Port Scriere Date	DATA_WRITE_ADDR	In	20	Adresa fizică pentru scriere.
	WRITE_EN	In	1	Activare scriere sincronă.
	LOAD_DATA	In	16	Datele ce urmează a fi scrise.

4.13 Unitatea de Afișare (SSD)

Modulul **SSD** este un dresor de periferice pur combinatoriu (cu un contor de ceas intern) responsabil cu afișarea conținutului registrelor pe display-ul cu șapte segmente (7-segment display). Deoarece display-ul de pe placa FPGA este, de obicei, multiplexat (o singură cifră este activă la un moment dat), acest modul folosește o tehnică de scanare rapidă pentru a da impresia că toate cifrele sunt aprinse simultan.

4.13.1 Design și Funcționalitate

Modulul primește o valoare de 16 biți (de la un registru intern al CPU, de exemplu AX) și o împarte în cele patru cifre hexazecimale necesare afișării.

1. **Contor de Scanare:** Un contor intern sincronizat cu ceasul principal generează o adresă de 2 biți (sel), utilizată pentru a comuta rapid între cele patru cifre (digits 0 la digits 3).
2. **Multiplexare Cifre:** La fiecare ciclu de scanare, un proces (muxCat) selectează un grup de 4 biți (un *nibble* hexazecimal) din valoarea de intrare de 16 biți.
3. **Activare Anod (An):** Un alt proces (muxAn) activează anodul corespunzător cifrei selectate.
4. **Decodificare Catod (Cat):** Logica combinatorie finală (with digit SELect) convertește valoarea de 4 biți în semnalele de 7 segmente necesare pentru a forma vizual cifra corespunzătoare (A, F, 5, E, etc.).

4.13.2 Interfața Modulului

Categorie	Port	Tip	Lățime	Descriere
Sincronizare	clk	In	1	Semnal de ceas principal (pentru contorul de scanare).
Intrare Date	digits	In	16	Valoarea Hexazecimală de afișat (ex: conținutul registrului AX).
Ieșiri	an	Out	8	Ieșiri pentru Anod (selectează cifra activă prin scanare).
	cat	Out	7	Ieșiri pentru Catod (codul segmentelor).

4.14 Generatorul de Puls Unic (MPG)

Modulul **MPG** (Mono Pulse Generator) este o componentă periferică cu rolul de a transforma o apăsare fizică, zgomotoasă și prelungită a unui buton (btn) într-un singur puls logic curat (enable) de lățime fixă. Acest lucru este vital pentru circuitele secvențiale (cum ar fi FSM-ul din Unitatea de Control) care ar trebui să avanseze o singură stare la fiecare apăsare.

4.14.1 Arhitectură și Scop

Designul modulului îndeplinește două funcții principale:

1. **Debouncing (Filtrare):** Eliminarea zgomotului generat de contactul mecanic al butonului, asigurând stabilitatea semnalului. Această funcție este realizată prin eșantionarea (sampling) semnalului de intrare la intervale mari de timp, utilizând un contor intern.
2. **Generarea Pulsului Unic:** Transformarea semnalului stabilizat într-un singur front logic pozitiv (un "Mono Pulse") la eliberarea sau apăsarea butonului.

4.14.2 Interfața Modulului

Categorie	Port	Tip	Lățime	Descriere
Sincronizare	clk	In	1	Semnalul de ceas al sistemului.
Intrare Fizică	btn	In	1	Semnalul brut de la butonul fizic (zgomotos).
Ieșire Logică	enable	Out	1	Semnal de ieșire curat, puls unic (trimis la ControlUnit).

5. Implementare

Acest capitol detaliază realizarea concretă a proiectului, făcând tranziția de la arhitectura modulară (Capitolul 4) la nivelul de sintaxă VHDL și circuite fizice. Aici sunt prezentate deciziile critice de cablaj care dau viață Datapath-ului, logica detaliată a Mașinii de Stări Finite (FSM) și procesele VHDL care implementează funcționalități complexe (cum ar fi scrierea selectivă pe 8 biți și logica de segmentare). De asemenea, sunt incluse și rezultatele sintezei pe FPGA și procedura de validare hardware a execuției programului de test.

5.1 Implementarea Modulului de Top-Level (test_env)

Implementarea modulului **test_env** se concentrează pe instanțierea și cablarea componentelor majore (**BIU, EU, Segmented_RAM, MPG, SSD**) și pe logica dedicată interfeței cu utilizatorul (afișarea datelor).

5.1.1 Logica de Afișare și Debug (SSD Formatting)

Logica de formatare a display-ului este implementată printr-un proces combinatoriu (SSD_Format_Process) care acționează ca un multiplexor mare, citind switch-urile (sw) pentru a selecta sursa de date care va fi trimisă display-ului.

- **Multiplexarea Surselor (VHDL Case):** Logica internă utilizează biții inferiori ai switch-urilor (sw(3 downto 0)) pentru a alege una din cele zece surse de debug disponibile (Registre, Flag-uri, Instrucțiuni sau poziții FIFO), ca în fragmentul VHDL următor:

```
-- Logica de selecție a datelor pentru display-ul cu 7 segmente (SSD)
case sw(3 downto 0) is
    when "0010"  => s_ssd_digits <= v_flags_display; -- Flag-uri
    when "0011"  => s_ssd_digits <= s_ax_out;        -- AX Register
    when "0111"  => s_ssd_digits <= s_q_out_0_fifo;  -- FIFO(0)
    when others  => s_ssd_digits <= s_instr_to_eu;   -- Default: Următoarea
instrucțiune
end case;
```

- **Afișarea Flag-urilor pe LED-uri:** Logica implementată asigură, de asemenea, că registrul de flag-uri (s_current_flags) este mapat direct la LED-urile de joasă ordine (led(3 downto 0)), oferind o metodă rapidă și directă de monitorizare a stării logice a procesorului.

5.2 Implementarea Unității de Interfață cu Magistrala (BIU)

Implementarea modulului **BIU** se concentrează pe definirea cablajului intern (rutarea semnalelor) și pe logica sincronă a Mașinii de Stări Finite (FSM) care gestionează ciclul de pre-fetch.

5.2.1 Logica Controlerului FSM (Procesul BIU_Controller_Process)

Logica FSM asigură că instrucțiunile sunt aduse din memorie (FETCH) și scrise în coada **FIFO** (o operație sincronă care durează două cicluri de ceas: FETCH_WAIT_1 și FETCH_WRITE), permițând **BIU** să opereze autonom.

- **Logica de Timp a Ciclului Fetch:** Tranzacția Fetch este structurată pe doi pași secvențiali. FETCH_WAIT_1 este starea de latență, urmată de FETCH_WRITE, unde are loc scrierea propriu-zisă în coadă și actualizarea IP.

- **Activarea Finală:** În starea FETCH_WRITE, FSM-ul implementează simultan cele două acțiuni critice VHDL care finalizează ciclul:

```
when FETCH_WRITE =>
    -- Activeaza scrierea in FIFO pentru instructiunea citita
    s_fifo_write_en    <= '1';
    -- Activeaza incrementarea Contorului de Program (IP <- IP + 2)
    s_increment_ip_en  <= '1';
    biu_state          <= IDLE;
```

- **Prioritatea semnalului FLUSH (Logica de Reset):** Implementarea acordă prioritate maximă semnalului FLUSH_FROM_EU. Logica VHDL este poziționată la începutul procesului FSM, asigurând că un salt resetează imediat ciclul de fetch:

```
elsif rising_edge(CLK) then
    if FLUSH_FROM_EU = '1' then
        -- Intrerupe FSM-ul si reseteaza logica de control
        biu_state          <= IDLE;
        s_fifo_write_en    <= '0';
        s_increment_ip_en  <= '0';
    else
        -- Continua executia FSM-ului normal (case biu_state is...)
```

5.2.2 Rutarea Semnalelor de Adresă și Date (VHDL Wiring)

Implementarea **BIU** acționează ca un centru de rutare, stabilind conexiunile fizice între sub-modulele sale și porturile externe.

- **Magistrala de Adrese (Fetch):** Ieșirea adresei fizice a instrucțiunii (s_instr_addr_out) de la **AddressCalculator** este conectată direct la portul extern INSTR_ADDR_OUT al **BIU**, realizând conexiunea fizică către Memoria **RAM**.

- **Cablajul Datelor (Pass-through): BIU** asigură că semnalele de scriere/citire a datelor inițiate de **EU** sunt transmise transparent la **RAM**. Aceasta include rutarea semnalelor de control (DATA_WRITE_EN_FROM_EU și DATA_TO_WRITE_FROM_EU) și a adresei de scriere (RAM_WRITE_ADDR_OUT) direct la **Memorie**, **BIU** neintervenind asupra acestora.

```
-- Logica VHDL de pass-through pentru operatiile de scriere:
RAM_WRITE_EN_OUT   <= DATA_WRITE_EN_FROM_EU;
RAM_LOAD_DATA_OUT  <= DATA_TO_WRITE_FROM_EU;
RAM_WRITE_ADDR_OUT <= s_data_read_addr_out; -- Adresa de date de la
AddressCalculator
```

5.3 Implementarea Registrelor de Segment (SegmentsReg)

Implementarea modulului **SegmentsReg** este realizată printr-un proces sincronizat (Register_Update_Process) care stochează cele cinci registre esențiale (CS, IP, DS, SS, ES). Logica se concentrează pe definirea valorilor inițiale ale segmentelor, care stabilesc cadrul de memorie pentru programul de test.

5.3.1 Logica de Inițializare și Actualizare IP

Modulul utilizează constante VHDL pentru a seta valorile de bază ale segmentelor la Reset. Procesul VHDL Register_Update_Process implementează o logică sincronă:

- **Inițializarea (RST):** La semnalul de reset, toate registrele sunt încărcate cu valorile predefinite de start, mapând logic segmentele la adresele fizice de bază ale memoriei alocate (ex: C_DS_INIT = x"0020" corespunde adresei fizice 00200h).

- **Actualizarea IP (IP Tracking):** Logica implementată permite actualizarea dinamică, la fiecare ciclu de ceas, doar a Contorului de Instrucțiuni (IP_REG), cu valoarea calculată de modulul **AddressCalculator** (NEW_IP_IN). Aceasta este singura valoare care se modifică secvențial în timpul execuției programului.

```
-- Fragment VHDL pentru inițializare și actualizare sincronă:
elsif rising_edge(CLK) then
    -- Actualizarea Contorului de Instrucțiuni (IP)
    IP_REG <= NEW_IP_IN;
end if;

-- Logica de reset:
if rst = '1' then
    DS_REG <= C_DS_INIT; -- x"0020"
    CS_REG <= C_CS_INIT; -- x"0000"
    -- ... și celelalte registre (SS, ES)
    IP_REG <= (others => '0');
```

5.3.2 Constantele de Inițializare

Constantele definite în corpul arhitecturii stabilesc punctele de intrare ale memoriei pentru procesor, care sunt vitale pentru încărcarea programului de test și a stivei.

Constantă	Valoare (Hex)	Adresă Fizică de Start	Rol
C_CS_INIT	x"0000"	00000h	Baza Segmentului de Cod (Program Start).
C_DS_INIT	x"0020"	00200h	Baza Segmentului de Date.
C_SS_INIT	x"0040"	00400h	Baza Segmentului de Stivă.
C_ES_INIT	x"0040"	00400h	Baza Segmentului Extra.

5.4 Implementarea Calculatorului de Adrese (AddressCalculator)

Modulul **AddressCalculator** este implementat ca un proces pur combinatoriu (Addr_Calc_Process), utilizând pachetele NUMERIC_STD și STD_LOGIC_UNSIGNED pentru a facilita operațiile aritmetice pe vectori de 16 și 20 de biți. Implementarea VHDL se concentrează pe evitarea întârzierilor prin realizarea tuturor calculelor de adresă într-un singur ciclu de ceas.

5.4.1 Logica de Calcul a Adresei Fizice (Segmentare)

Implementarea logicii de segmentare se realizează prin operații de concatenare (shift logic) urmate de adunare, conform formulei Segment << 4 + Offset.

- **Deplasarea Segmentului:** Registrul segment (CS_IN sau DS_IN) este deplasat la stânga cu 4 biți prin concatenarea a patru zerouri. Această tehnică VHDL simulează înmulțirea cu 16, necesară pentru a obține adresa de bază a segmentului pe 20 de biți.

```
-- Fragment VHDL pentru calculul bazei segmentului de cod (20 biți):
v_cs_segment_base_20 := unsigned(CS_IN & "0000");

-- Calculul adresei fizice CS:IP
CURRENT_PHYSICAL_ADDR_OUT <= std_logic_vector(v_cs_segment_base_20 +
v_current_ip_20);
```

- **Adresarea Datelor:** Adresarea datelor (DATA_PHYSICAL_ADDR_OUT) este implementată similar, utilizând registrul DS_IN și offset-ul primit de la EU (DATA_OFFSET_IN).

5.4.2 Logica de Control a Contorului de Instrucțiuni (IP)

Procesul VHDL include o logică de selecție finală care stabilește următoarea valoare a Contorului de Instrucțiuni (NEXT_IP_OUT), rezolvând astfel conflictele dintre avansul secvențial și saltul necondiționat:

- **Avans Secvențial:** NEXT_IP_OUT este incrementat cu constanta 2 (v_ip_in_16 + 2), deoarece instrucțiunile sunt de lungime fixă (16 biți).
- **Salt (Jump):** Dacă semnalul JUMP_EN este activat, valoarea NEXT_IP_OUT preia adresa țintă furnizată de **EU**. Logica asigură că JUMP_EN are prioritate asupra incrementării secvențiale a IP.

```
-- Fragment VHDL pentru selecția finală a IP:
if JUMP_EN = '1' then
    NEXT_IP_OUT <= std_logic_vector(v_jump_next_ip_16); -- Adresa țintă
elsif INCREMENT_IP_EN = '1' then
    NEXT_IP_OUT <= std_logic_vector(v_sequential_next_ip_16); -- IP + 2
else
    NEXT_IP_OUT <= IP_IN; -- Menține starea curentă
end if;
```

5.5 Implementarea Cozii FIFO de Instrucțiuni (CFIFO)

Implementarea modulului **CFIFO** (Coada de Instrucțiuni) este realizată printr-un singur proces sincron (Queue_Control_Process) care gestionează memoria internă (Q_MEM) și starea cozii (pointerii Q_HEAD și Q_TAIL și contorul Q_COUNT).

5.5.1 Logica Sincronă a Pointerilor

Logica VHDL pentru citire, scriere și resetare este implementată strict pe frontul crescător al ceasului (rising_edge(CLK)), asigurând integritatea stării. Procesul utilizează variabile interne (V_HEAD_NEXT, V_TAIL_NEXT, V_COUNT_NEXT) pentru a calcula noua stare a cozii înainte de a o aplica registrelor sincronice la sfârșitul ciclului.

- **Pointerii Circulari:** Deoarece coada are o adâncime fixă (Q_DEPTH = 3), implementarea necesită utilizarea aritmeticii modulo pentru a reseta pointerii la zero după ce ajung la capătul cozii, realizând astfel o structură de coadă circulară.

```
-- Fragment VHDL pentru avansarea pointerului de citire (Q_HEAD)
if READ_EN = '1' and Q_COUNT > 0 then
    V_HEAD_NEXT := (Q_HEAD + 1) mod Q_DEPTH; -- Aritmetica circulara
    V_COUNT_NEXT := V_COUNT_NEXT - 1;
end if;

-- Fragment VHDL pentru avansarea pointerului de scriere (Q_TAIL)
if WRITE_EN = '1' and V_COUNT_NEXT < Q_DEPTH then
    Q_MEM(Q_TAIL) <= INSTRUCTION_IN;
    V_TAIL_NEXT   := (Q_TAIL + 1) mod Q_DEPTH;
    V_COUNT_NEXT := V_COUNT_NEXT + 1;
end if;
```

5.5.2 Logica de Prioritate (Flush)

O decizie critică de implementare este acordarea de prioritate maximă semnalului FLUSH (golirea cozii), care este activat de Unitatea de Execuție la un salt reușit. Logica VHDL resetează imediat toți pointerii la zero (poziția de start), ignorând orice operație de citire sau scriere care ar putea avea loc în același ciclu de ceas.

```
-- Fragment VHDL pentru logica de prioritate a semnalului FLUSH:
elsif rising_edge(CLK) then
    -- PRIORITATEA MAXIMA
    if FLUSH = '1' then
        Q_HEAD  <= 0;
        Q_TAIL  <= 0;
        Q_COUNT <= 0;
    else
        -- Logica normala de citire/scriere (Read/Write)
        -- ...
```

5.5.3 Ieșiri Combinatorii de Stare

Stările critice ale cozii (FULL și EMPTY) sunt realizate prin logică combinatorie (asincronă) care compară instantaneu contorul de ocupare (Q_COUNT) cu valoarea maximă a cozii (Q_DEPTH) sau cu zero, furnizând feedback imediat către **BIU** și **EU**.

```
FULL  <= '1' when Q_COUNT = Q_DEPTH else '0';
EMPTY <= '1' when Q_COUNT = 0 else '0';
```

5.6 Implementarea Unității de Execuție (EU)

Implementarea modulului **EU** realizează structura fizică a Datapath-ului. Deși mare parte din funcționalitate este delegată sub-modulelor, fișierul de top-level EU.vhd conține logica combinatorie critică pentru rutarea datelor (Multiplexare) și extensia operanzilor.

5.6.1 Logica de Selecție a Operanzilor ALU (Multiplexare)

Pentru a alimenta **Unitatea Aritmetico-Logică** cu datele corecte, implementarea utilizează structuri de decizie VHDL (when...else și with...select) care acționează ca multiplexoare hardware.

- **Adaptarea Operandului A (8 vs 16 biți):** Deoarece **ALU** operează nativ pe 16 biți, dar procesorul suportă operații pe 8 biți (ex: ADD AL, BL), implementarea include o logică de extensie cu zerouri. Dacă semnalul de control s_cu_size_sel indică o operație pe octeți ('1'), datele de 8 biți din registru sunt concatenate cu 8 zerouri în partea superioară.

```
-- Fragment VHDL: Multiplexorul pentru Intrarea A a ALU
s_alu_in_a <= s_reg_data_a_16 when s_cu_size_sel = '0' else
              ("00000000" & s_reg_data_a_8);
```

- **Selecția Sursei pentru Operandul B:** Al doilea operand al **ALU** poate proveni din trei surse diferite. Unitatea de Control setează semnalul s_cu_mux_b_sel, iar logica de implementare rutează calea corespunzătoare:

  1. **Registru (MUX_B_REGS):** Valoarea din Portul B al RegFile (adaptată similar cu Operandul A).

2. **Memorie (MUX_B_MEM):** Datele citite de la intrarea MEM_DATA_IN (pentru instrucțiuni precum ADD AX, [BX]).

3. **Imediat (MUX_B_IMM):** O valoare constantă extrasă direct din codul instrucțiunii (pentru ADI, LIR).

```
-- Fragment VHDL: Multiplexorul Principal pentru Intrarea B
with s_cu_mux_b_sel select
    s_alu_in_b <=
        s_reg_data_b_for_alu  when MUX_B_REGS,
        MEM_DATA_IN           when MUX_B_MEM,
        s_extended_immediate  when MUX_B_IMM,
        (others => '0')       when others;
```

5.6.2 Logica de Extensie a Valorilor Imediate

Pentru instrucțiunile care conțin constante (valori imediate), cum ar fi LIR (Load Immediate) sau ADI (Add Immediate), valoarea este stocată pe doar 7 biți în interiorul instrucțiunii. Implementarea VHDL extrage acești biți și realizează o extensie de semn la 16 biți folosind funcția resize din biblioteca NUMERIC_STD, permițând utilizarea numerelor negative în operații imediate.

```
-- Fragment VHDL: Extragerea si extensia valorii imediate
v_imm7 := s_cu_ir(6 downto 0); -- Extrage ultimii 7 biti din IR
-- Extensie de semn la 16 biti
s_extended_immediate <= std_logic_vector(resize(signed(v_imm7), 16));
```

5.6.3 Selecția Flag-urilor Corecte

Deoarece **ALU** calculează simultan două seturi de flag-uri (pentru rezultate de 8 și 16 biți), implementarea **EU** trebuie să decidă care set este salvat în **FlagsRegister**. Această decizie se bazează pe același semnal de dimensiune (SIZE_SEL) utilizat în restul Datapath-ului.

```
-- Logica de rutare a flag-urilor catre registrul de stare
s_flags_to_reg <= s_flags_out_8 when s_cu_size_sel = '1' else
                  s_flags_out_16;
```

5.7 Implementarea Unității de Control (ControlUnit)

Modulul **ControlUnit** este implementat ca un proces sincron care conține Mașina de Stări Finite (FSM) a procesorului multi-ciclu. Implementarea sa se bazează pe decodificarea pe 5 biți a opcodului și pe generarea simultană a tuturor semnalelor de control.

5.7.1 Decodificarea Instrucțiunilor și Extracția Câmpurilor

Logica combinatorie inițială extrage toate câmpurile necesare direct din Registrul de Instrucțiuni (IR), stocat intern, inclusiv registrele de operanzi (s_ddd, s_sss), bitul de lățime (s_size) și adresa de salt (s_jmp_addr11).

```
-- Fragment VHDL: Extragerea campurilor instrucțiunii (Logica combinatorie)
s_opcode     <= IR(15 downto 11);
s_size       <= IR(10);
s_ddd        <= IR(9 downto 7);
s_sss        <= IR(6 downto 4);
s_jmp_addr11 <= IR(10 downto 0);
```

5.7.2 Logica FSM și Microcodul Secvențial

Procesul VHDL sincron (process(CLK, RST)) gestionează FSM-ul cu șase stări, controlând exact ce semnale sunt activate în fiecare ciclu de execuție. Logica FSM este implementată printr-o structură principală case state is... care include o structură case s_opcode is... (microcodul) în interiorul stărilor s_DECODE și s_EXECUTE.

Logica Stării s_DECODE (Decizie și Inițializare)

În starea s_DECODE, FSM-ul analizează opcodul și decide calea de tranziție (EXECUTE, MEM_READ, JUMP) și, simultan, setează selectorii MUX și codul de operație ALU pentru ciclul următor.

```
when s_DECODE =>
    case s_opcode is
        -- Operații ALU Reg-Reg (ex: ADD)
        when "00000" =>
            if s_t_bit = '0' then -- Reg-Reg mode
                ALU_OP_SEL       <= OP_ADD;
                ALU_IN_B_MUX_SEL <= MUX_B_REGS;
                -- REG_READ_ADDR_B este suprapus (override) pentru a citi
s_addr6(2 downto 0)
                state <= s_EXECUTE;
            else -- Memorie mode (necesita un ciclu in plus)
                state <= s_MEM_READ;
            end if;
        -- Operație Memorie (ex: LDR)
        when "01000" =>
            -- Calculeaza offset-ul de memorie in DATA_OFFSET_OUT
            state <= s_MEM_READ;
        -- Salturi
        when "10000" | "10001" | "10010" =>
            state <= s_JUMP;
        -- ...
```

Logica Stării s_EXECUTE (Finalizare și Scrierea Datelor)

Starea s_EXECUTE asigură că rezultatul **ALU** este scris în registrul corect și că flag-urile sunt actualizate condițional:

- **Scrierea Registrului:** Semnalul REG_WRITE_EN este activat pentru toate instrucțiunile care nu sunt CMP (Comparație).
- **Actualizarea Flag-urilor:** Semnalul FLAG_REG_WRITE_EN este activat numai dacă instrucțiunea executată (ex: ADD, SUB, CMP, Logic) modifică starea, fiind dezactivat pentru operațiile de transfer pur (MOV, LIR, LDR).

Logica Stării s_JUMP (Evaluarea Condițiilor)

Starea s_JUMP este responsabilă de evaluarea condițiilor logice bazate pe Flag-uri și de emiterea semnalului de salt către BIU.

- **Evaluare Flag-uri:** Bitul Z_FLAG (Zero Flag) este extras din CURRENT_FLAGS(0) și utilizat pentru a decide saltul.

- **Comanda de Salt:** Dacă condiția este îndeplinită, Unitatea de Control trimite două semnale critice către **BIU**: BIU_JUMP_EN (pentru a schimba IP) și BIU_FLUSH (pentru a goli coada FIFO de instrucțiuni invalide).

```
when s_JUMP =>
    -- ... seteaza BIU_JUMP_TARGET ...

    if s_opcode = "10000" then -- JMP (Necondiționat)
        BIU_JUMP_EN <= '1';
        BIU_FLUSH   <= '1';
    elsif s_opcode = "10001" and Z_FLAG = '1' then -- JZ (Salt dacă Zero
Flag este activ)
        BIU_JUMP_EN <= '1';
        BIU_FLUSH   <= '1';
    elsif s_opcode = "10010" and Z_FLAG = '0' then -- JNZ (Salt dacă Zero
Flag este inactiv)
        BIU_JUMP_EN <= '1';
        BIU_FLUSH   <= '1';
    end if;
    state <= s_IDLE;
```

5.8 Implementarea Fișierului de Registre (RegFile)

Implementarea modulului **RegFile** este realizată printr-un proces sincron pentru scriere și procese combinatorii pentru citire, având ca scop replicarea logicii complexe de adresare pe octeți specifică arhitecturii 8086.

5.8.1 Logica de Scriere Selectivă (Process Write_Process)

Procesul Write_Process este sincronizat cu ceasul și gestionează logica esențială de scriere condiționată. Această logică determină dacă se scrie în întregul registru (16 biți) sau doar într-un octet selectat (8 biți), utilizând semnalul SIZE_SEL și biții de adresă.

- **Sinteza Logicii pe Octeți:** Dacă SIZE_SEL = '1', codul VHDL utilizează o structură case ADDR_WR(2 downto 1) is pentru a selecta registrul de bază (AX, BX, CX, DX) și o structură if/else pentru a selecta octetul (High/Low) pe baza bitului ADDR_WR(0).

```
-- Fragment VHDL pentru Scriere Selectivă de 8 biți:
elsif rising_edge(CLK) then
    if WR_EN = '1' then
        if SIZE_SEL = '1' then
            -- 8-bit write
            case ADDR_WR(2 downto 1) is
                when "00" => -- AX
                    if ADDR_WR(0) = '1' then -- Low byte (AL)
                        AX_REG(7 downto 0) <= DATA_WR(7 downto 0);
                    else -- High byte (AH)
                        AX_REG(15 downto 8) <= DATA_WR(7 downto 0);
                    end if;
                -- ... logica se repeta pentru BX, CX, DX
                when "100" | "101" | "110" | "111" =>
                    -- Registrele de pointeri (SP, BP, SI, DI) sunt scrise
intotdeauna pe 16 biti
```

```
                    -- deoarece ADDR_WR(0) este ignorat pentru aceste
adrese
                    SP_REG <= DATA_WR; -- Exemplu pentru SP_REG
                    -- ...
```

- **Registrele de Pointeri:** Deși ADDR_WR este pe 3 biți, registrele de pointeri (SP, BP, SI, DI) sunt tratate ca fiind adresabile exclusiv pe 16 biți, scrierea făcându-se întotdeauna pe întreaga lățime a registrului.

5.8.2 Implementarea Căilor de Citire Combinatorii

Porturile de citire sunt implementate prin logică combinatorie (asincronă), asigurând că datele sunt disponibile imediat pentru **ALU**.

- **Citirea pe 16 biți:** Este realizată printr-un multiplexor simplu (logica with ADDR_RD_A select...) care rutează valoarea completă a registrului solicitat către ieșirea de 16 biți (DATA_RD_A_16).
- **Citirea pe 8 biți:** Aceasta implică o logică de selecție dublă. Sunt calculate în paralel ambii octeți (High și Low) pentru registrele AX-DX, iar selectorul final folosește ADDR_RD_A(0) pentru a alege între Low Byte (AL, BL) și High Byte (AH, BH).

```
-- Fragment VHDL pentru Citirea pe 8 biți (Portul A):
-- Selecteaza care registru (AX, BX, CX, DX) se citeste:
s_rd_a_phys_addr <= ADDR_RD_A(2 downto 1);

-- Logic combinatorie care alege octetul Low/High pe baza adresei:
DATA_RD_A_8 <= s_rd_a_8_high when (SIZE_SEL = '1' and ADDR_RD_A(0) = '0')
else -- AH, BH, etc.
               s_rd_a_8_low  when (SIZE_SEL = '1' and ADDR_RD_A(0) = '1')
else -- AL, BL, etc.
               (others => 'X');
```

5.9 Implementarea Unității Aritmetico-Logice (ALU16b)

Implementarea **ALU** se concentrează pe o abordare structurală pentru Nucleul Aritmetic și o abordare comportamentală/combinatorie pentru Logica de Control și Flag-uri.

5.9.1 Construcția Nucleului Aritmetic (Structural VHDL)

Nucleul Aritmetic este implementat ca un Lanț de Sumatoare (Ripple-Carry Chain) pe 16 biți, prin instanțierea repetitivă a modulului pe 1 bit (OneBitALU). Această metodă definește explicit căile de propagare a bitului de transport (Carry).

- **Instanțierea în Lanț:** Se utilizează o instrucțiune VHDL generate (buclă for) pentru a crea 16 celule, unde ieșirea de transport (C_OUT) a celulei I devine intrarea de transport (C_IN) a celulei I+1.
- **Logica de Scădere:** Scăderea (OP_SUB) este realizată prin inversarea Operandului B (ALU_SUB_SEL = '1') și setarea bitului de transport inițial (ALU_CARRY_IN) la '1' (realizând astfel adunarea cu complementul față de doi: A + ~B + 1).

5.9.2 Logica de Selecție și Deplasare

Rezultatul final (ALU_OUT) este ales dintr-un multiplexor combinatoriu pe 16 biți.

```
-- Fragment VHDL pentru MUX-ul final al rezultatului (Logica combinatorie)
```

```
with ALU_OP_SEL select
    ALU_OUT <=
        ALU_CORE_RESULT when OP_ADD | OP_SUB | OP_INC | OP_DEC | OP_AND |
OP_OR | OP_XOR | OP_NOT,
        SHIFTER_OUT     when OP_SHL | OP_SHR,
        DATA_IN_B       when OP_PASS_B, -- Pentru transfer de date (MOV,
LIR, LDR)
        (others => '0') when others;
```

- **Blocul de Shiftare:** Deplasările (OP_SHL, OP_SHR) sunt implementate comportamental (shift_left / shift_right din NUMERIC_STD), fiind selectate doar atunci când Codul de Operație o cere.

5.9.3 Calculul Dual al Flag-urilor (Logica Combinatorie)

ALU utilizează două procese combinatorii separate pentru a calcula Flag-urile pentru operațiile de 8 biți și 16 biți, deoarece biții de Overflow și Carry diferă.

- **Flag-ul Zero (Z):** Calculat prin verificarea dacă toți biții rezultatului sunt zero (integral pentru Z16, doar pe biții 7 downto 0 pentru Z8).
- **Flag-ul de Carry (C):** c16 este bitul de transport al poziției 15 (ALU_CARRY(15)). c8 este bitul de transport al poziției 7 (ALU_CARRY(7)).
- **Flag-ul de Overflow (O):** Este implementat prin verificarea bitului de transport final vs. bitul de transport precedent (ex: O16 = C15 | C14).

```
-- Fragment VHDL pentru Overflow Flag (16 biți):
-- O Flag este setat daca carry-in la bitul 15 (C14) difera de carry-out
(C15)
o16 <= ALU_CARRY(15) xor ALU_CARRY(14);
```

5.10 Implementarea Celulei de Calcul pe 1 Bit (OneBitALU)

Modulul **OneBitALU** este realizat ca o componentă pur combinatorie, unde funcțiile aritmetice și logice de bază sunt implementate direct prin ecuații booleene. Acest modul definește modul în care **ALU**-ul general gestionează operațiile de adunare, scădere și logică la nivelul cel mai de jos.

5.10.1 Logica Aritmetică de Bază (Full Adder/Subtractor)

Implementarea utilizează semnalul SUB_SEL pentru a realiza scăderea. Această logică este aplicată permanent (concurrent), în afara procesului FSM, deoarece este pur combinatorie.

- **Inversarea pentru Scădere:** Scăderea este realizată prin adunarea cu complementul față de unu, unde Operandul B este inversat logic (prin XOR) dacă SUB_SEL este activ.

```
B_MODIFIED <= B_IN xor SUB_SEL;
```

- **Generarea Suma și Transport:** Bitul de sumă (SUM_BIT) și bitul de transport (C_SUM) sunt generate folosind ecuațiile standard ale unui sumator complet (Full Adder):

```
SUM_BIT <= A_IN xor B_MODIFIED xor C_IN;

C_SUM   <= (A_IN and B_MODIFIED) or (C_IN and A_IN) or (C_IN and
B_MODIFIED);
```

5.10.2 Multiplexorul de Funcție (Logica de Selecție)

Logica principală a celulei este un multiplexor VHDL (case FUNC_SEL is...) care alege între rezultatul aritmetic (SUM_BIT, C_SUM) și rezultatele operațiilor logice (AND, OR, XOR).

FUNC_SEL	Operație	Sursă Rezultat	Sursă Carry (C_OUT)
"00"	Adunare/Scădere	SUM_BIT	C_SUM (Bit de transport/împrumut)
"01"	AND	A_IN and B_IN	0 (Carry nu este folosit)
"10"	OR	A_IN or B_IN	0
"11"	XOR	A_IN xor B_IN	0

Această structură implementează, efectiv, o celulă ALU multifuncțională, care este baza pentru construirea lanțului de 16 biți în modulul de nivel superior.

5.11 Implementarea Registrului de Flag-uri (FlagsRegister)

Modulul **FlagsRegister** este un registru sincron de 4 biți implementat printr-un singur proces VHDL, având rolul de a reține starea procesorului (Flag-urile S, O, C, Z) între ciclurile de execuție.

5.11.1 Logica de Actualizare Sincronă

Actualizarea Flag-urilor este controlată de două semnale principale:

1. **RST:** Resetează toate flag-urile la 0000 la pornire.
2. **FLAG_WRITE_EN:** Activează scrierea noilor valori (FLAGS_IN) primite de la ALU, dar numai la frontul crescător al ceasului (rising_edge(CLK)).

Acest design asigură că, deși **ALU** generează continuu valori noi de flag-uri, acestea sunt scrise în registru doar în ciclul s_EXECUTE și doar pentru instrucțiunile care modifică flag-urile (evitând, de exemplu, actualizarea la o simplă instrucțiune MOV sau LDR).

```
-- Fragment VHDL din Procesul de Actualizare a Flag-urilor:
Flags_Update_Process: process (CLK, RST)
begin
    if RST = '1' then
        s_flags_reg <= "0000"; -- Reset
    elsif rising_edge(CLK) then
        if FLAG_WRITE_EN = '1' then
            -- Scriere sincronă, controlată de Unitatea de Control
            s_flags_reg <= FLAGS_IN;
        end if;
    end if;
end process Flags_Update_Process;
```

5.12 Implementarea Unității de Memorie Segmentată (Segmented_RAM)

Modulul **Segmented_RAM** implementează memoria internă a sistemului, esențială pentru stocarea programului și a datelor. Logica sa se concentrează pe o implementare eficientă a memoriei cu acces multi-port și pe logica de mapare a adresei de 20 de biți la matricea internă.

5.12.1 Inițializarea Programului (Funcția Impură InitRam)

Memoria internă (MEMORY) este populată la momentul sintezei (Initialization) cu programul de test pre-compilat, simulând o memorie ROM/Flash. Acest lucru este realizat prin utilizarea unei funcții impure VHDL (InitRam).

- **Alocarea Statică:** Modulul definește constantele pentru segmente (CODE_WORDS = 256, DATA_WORDS = 256), stabilind adâncimea totală a matricei la 514 cuvinte (RAM_SIZE_WORDS).
- **Încărcarea Hex:** Funcția InitRam încarcă programul de test (coduri hexazecimale, ex: X"3831") în zona alocată Code Segment, începând cu indexul zero.

5.12.2 Logica de Conversie a Adresei (Address Decoder)

Deoarece adresele fizice primite de la **BIU** sunt pe 20 de biți (adresă de octet), iar memoria internă este organizată pe 16 biți (adresă de cuvânt), este necesară o conversie a indicilor. Procesul RAM_Index_Conversion realizează această mapare.

- **Maparea Adresei:** Logica implementată ignoră bitul cel mai puțin semnificativ (LSB), realizând o împărțire implicită la 2, convertind adresa de octet în adresa de cuvânt (indexul matricial).

```
-- Fragment VHDL: Maparea adresei de 20 biți la indexul matricial intern
RAM_INDEX_INSTR <= to_integer(unsigned(INSTR_ADDR(19 downto 1)));
```

- **Validarea Adresei:** Procesul include, de asemenea, o logică de validare care verifică dacă adresa calculată depășește dimensiunea maximă alocată (MAX_INDEX), prevenind astfel depășirea limitelor memoriei.

5.12.3 Procesul de Acces Multi-Port

Memoria implementează accesul dual (citire date vs. scriere date vs. citire instrucțiuni) prin procese sincronice și combinatorii:

1. **Portul de Citire Date (DATA_READ):** Este realizat printr-un proces combinatoriu (asincron). Aceasta asigură că datele sunt disponibile imediat pentru EU, fără latența unui ciclu de ceas.
2. **Portul de Scrierea Datelor și Citirea Instrucțiunilor (Sincron):** Procesul RAM_Access_Process este sincronizat de CLK și gestionează două acțiuni critice:
    - **Scrierea:** Se realizează la frontul crescător al ceasului, activată de semnalul WRITE_EN.
    - **Citirea Instrucțiunilor:** Valoarea este preluată din memorie și stocată într-un registru intern (INSTR_OUT_REG) pe frontul crescător al ceasului. Acest lucru conferă portului de instrucțiuni un comportament *latch-uit/sincron*, vital pentru temporizarea corectă a ciclului Fetch al BIU.

5.13 Implementarea Unității de Afișare (SSD)

Modulul **SSD** este implementat ca un dresor de periferice cu rol de multiplexare în timp, esențial pentru afișarea datelor de 16 biți pe un display fizic.

5.13.1 Logica de Scanare și Contorul Sincron

Logica de scanare asigură că display-ul este rapid reîmprospătat. Acest lucru se realizează printr-un contor sincron (counter) care este incrementat la fiecare ciclu de ceas.

- **Generarea Adresei de Scanare:** Biții superiori ai contorului (cnt(15 downto 14)) formează adresa de scanare (sel), care controlează care cifră (nibble) din intrarea de 16 biți este afișată.
- **Decizia de Multiplexare:** Procesele combinatorii (muxCat și muxAn) folosesc această adresă de 2 biți (sel) pentru a alege simultan atât datele (digit), cât și anodul corespunzător.

```
-- Fragment VHDL: Procesul de selecție a anodului (An)
muxAn : process(sel)
begin
    case sel is
        when "00" => an <= "11111110"; -- Activeaza AN0 (cifra 0)
        when "01" => an <= "11111101"; -- Activeaza AN1 (cifra 1)
        -- ...
    end case;
end process;
```

5.13.2 Logica de Decodificare Hexa-7 Segmente

Decodificarea Catodului (cat) este realizată printr-o logică pur combinatorie (with digit SELect), care mapează fiecare cifră hexazecimală (0-F) la combinația de 7 segmente necesară pentru a o ilumina pe display.

- **Maparea Datelor:** Procesul muxCat selectează grupul de 4 biți corespunzător (ex: digits(15 downto 12)) și îl rutează către semnalul digit.
- **Tranziția Finală:** Instrucțiunea with digit SELect convertește valoarea de 4 biți în codul de 7 biți, care este trimis către pinul cat.

```
-- Fragment VHDL pentru Decodificarea Catodului
with digit SELect
    cat <= "1000000" when "0000",   -- 0
           "1111001" when "0001",   -- 1
           "0000000" when "1000",   -- 8
           "0001110" when "1111",   -- F
           (others => 'X') when others;
```

5.14 Implementarea Generatorului de Puls Unic (MPG)

Implementarea modulului **MPG** se bazează pe circuite secvențiale pentru filtrare și pe logică combinatorie pentru generarea pulsului unic. Această componentă este crucială pentru a asigura că FSM-ul Unității de Execuție (**EU**) avansează o singură stare la o singură apăsare a butonului.

5.14.1 Logica de Filtrare (Debouncing)

- **Contorul Sincron (cnt_int):** Modulul utilizează un contor pe 18 biți care este incrementat la fiecare ciclu de ceas. Acesta servește ca bază de timp pentru a măsura durata zgomotului mecanic al butonului.
- **Eșantionarea (Sampling):** Semnalul fizic al butonului (btn) este eșantionat și înregistrat în Q1 doar atunci când contorul atinge valoarea sa maximă ("111111111111111111"). Această eșantionare la intervale mari ignoră stările intermediare și zgomotoase ale butonului, stabilizând semnalul.

```
-- Fragment VHDL pentru eșantionarea (sampling) semnalului btn:
if clk'event and clk='1' then
    if cnt_int(17 downto 0) = "111111111111111111" then
```

```
        Q1 <= btn; -- Q1 primește semnalul btn filtrat
    end if;
end if;
```

5.14.2 Generarea Pulsului Unic (Mono Pulse)

Semnalul stabilizat din Q1 este trecut printr-un registru de deplasare (shift register) sincron pe 3 biți (Q1, Q2, Q3) pentru a detecta tranziția de la starea "apasat" la starea "eliberat" sau invers.

- **Detectarea Frontului:** Ieșirea finală (enable) este generată printr-o combinație logică care produce un puls activ *doar* pentru un singur ciclu de ceas, atunci când semnalul se schimbă.

```
-- Logica sincronă de deplasare (Shift Register):
process(clk)
begin
    if clk'event and clk='1' then
        Q2 <= Q1;
        Q3 <= Q2;
    end if;
end process;

-- Logica combinatorie finală pentru pulsul unic:
enable <= Q2 and (not Q3);
```

Această formulă (Q2 AND NOT Q3) detectează momentul exact în care semnalul filtrat (Q2) este activ, dar starea sa din ciclul precedent (Q3) era inactivă, generând pulsul unic care avansează FSM-ul.

6. Testare și Validare

Acest capitol prezintă metodologia de verificare a funcționalității procesorului 8086 implementat, demonstrând corectitudinea execuției instrucțiunilor și integritatea fluxului de date. Procesul de validare s-a realizat exclusiv prin testare hardware directă pe platforma FPGA, utilizând un program de test pre-încărcat în memoria ROM simulată.

6.1 Metodologia de Testare (Single Stepping)

Validarea s-a bazat pe tehnica Execuției Pas-cu-Pas (Single Stepping) la nivel de instrucțiune, permițând izolarea efectului final al fiecărei comenzi asupra registrelor și memoriei.

6.1.1 Configurația de Test Hardware

Testarea a fost realizată utilizând resursele periferice ale plăcii de dezvoltare FPGA, mapate în modulul test_env:

- **Controlul Execuției:** Butonul central (btn(0)) a fost utilizat ca generator de ceas manual (STEP_BUTTON_IN), avansând procesorul cu exact o instrucțiune completă la fiecare apăsare.
- **Monitorizarea Stării:** Switch-urile (sw) au servit ca selector pentru multiplexorul de afișare, permițând vizualizarea în timp real a conținutului registrelor critice (AX, BX, CX, DX), a cozii sau a instrucțiunii curente.
- **Feedback Imediat:** LED-urile au fost utilizate pentru a afișa starea curentă a Registrului de Flag-uri (ZF, CF, SF, OF), oferind o confirmare vizuală instantanee a rezultatelor operațiilor aritmetice și a deciziilor de salt.

6.2 Scenariul de Testare (Programul de Referință)

Validarea completă a procesorului s-a realizat prin execuția unui program de test complex, conceput pentru a exercita toate categoriile funcționale implementate: transfer de date, aritmetică, logică și control al fluxului.

6.2.1 Structura Programului

Programul a fost scris inițial în limbaj de asamblare, apoi tradus manual în cod mașină hexazecimal și încărcat în zona de Code Segment a memoriei prin funcția InitRam. Fluxul de execuție a fost planificat și verificat în prealabil folosind un tabel de trasare (Excel Trace Table), care a servit ca referință de aur ("Golden Model") pentru validarea rezultatelor hardware.

Tabelul de trasare a definit starea așteptată a registrelor și flag-urilor după fiecare instrucțiune, permițând detectarea oricărei discrepanțe.

Id. Instr.	Instr.	Instr. Hex.	AX	AH	AL	BX	BH	BL	CX	CH	CL	DX	DH	DL	S	O	C	Z
0	LIR AX, 49	3831	49[0031]	0[00]	49[31]	0[0000]	0[00]	0[00]	0[0000]	0[00]	0[00]	0[0000]	0[00]	0[00]	0	0	0	0
1	LIR BL, 12	3D8C	49[0031]	0[00]	49[31]	12[000C]	0[00]	12[0C]	0[0000]	0[00]	0[00]	0[0000]	0[00]	0[00]	0	0	0	0
2	LIR CX, 4	3904	49[0031]	0[00]	49[31]	12[000C]	0[00]	12[0C]	4[0004]	0[00]	4[04]	0[0000]	0[00]	0[00]	0	0	0	0
3	LIR DH, 2	3F02	49[0031]	0[00]	49[31]	12[000C]	0[00]	12[0C]	4[0004]	0[00]	4[04]	512[0200]	2[02]	0[00]	0	0	0	0
4	INC DX	1980	49[0031]	0[00]	49[31]	12[000C]	0[00]	12[0C]	4[0004]	0[00]	4[04]	513[0201]	2[02]	1[01]	0	0	0	0
5	DEC AH	2400	65,329[FF31]	255[FF]	49[31]	12[000C]	0[00]	12[0C]	4[0004]	0[00]	4[04]	513[0201]	2[02]	1[01]	1	0	1	0
6	ADI AX, 2	1002	65,331[FF33]	255[FF]	51[33]	12[000C]	0[00]	12[0C]	4[0004]	0[00]	4[04]	513[0201]	2[02]	1[01]	1	0	0	0
7	ADD DX, CX	0184	65,331[FF33]	255[FF]	51[33]	12[000C]	0[00]	12[0C]	4[0004]	0[00]	4[04]	517[0205]	2[02]	5[05]	0	0	0	0
8	SUB AX, CX	0804	65,327[FF2F]	255[FF]	47[2F]	12[000C]	0[00]	12[0C]	4[0004]	0[00]	4[04]	517[0205]	2[02]	5[05]	1	0	1	0
9	STD AX, 0	4800	65,327[FF2F]	255[FF]	47[2F]	12[000C]	0[00]	12[0C]	4[0004]	0[00]	4[04]	517[0205]	2[02]	5[05]	1	0	1	0
10	LDR BX, 0	4080	65,327[FF2F]	255[FF]	47[2F]	65,327[FF2F]	255[FF]	47[2F]	4[0004]	0[00]	4[04]	517[0205]	2[02]	5[05]	1	0	1	0
11	AND AH, BH	5420	65,327[FF2F]	255[FF]	47[2F]	65,327[FF2F]	255[FF]	47[2F]	4[0004]	0[00]	4[04]	517[0205]	2[02]	5[05]	1	0	0	0
12	OR AH, AL	5C10	65,327[FF2F]	255[FF]	47[2F]	65,327[FF2F]	255[FF]	47[2F]	4[0004]	0[00]	4[04]	517[0205]	2[02]	5[05]	1	0	0	0
13	XOR AX, BX	6010	0[0000]	0[00]	0[00]	65,327[FF2F]	255[FF]	47[2F]	4[0004]	0[00]	4[04]	517[0205]	2[02]	5[05]	0	0	0	1
14	NOT BH	6D00	0[0000]	0[00]	0[00]	47[002F]	0[00]	47[2F]	4[0004]	0[00]	4[04]	517[0205]	2[02]	5[05]	0	0	0	1
15	NOT CX	6900	0[0000]	0[00]	0[00]	47[002F]	0[00]	47[2F]	65,531[FFFB]	255[FF]	251[FB]	517[0205]	2[02]	5[05]	1	0	0	0
16	SHL DX, 4	7190	0[0000]	0[00]	0[00]	47[002F]	0[00]	47[2F]	65,531[FFFB]	255[FF]	251[FB]	8572[2050]	32[20]	80[50]	0	0	0	0
17	SHR DH, 2	7F08	0[0000]	0[00]	0[00]	47[002F]	0[00]	47[2F]	65,531[FFFB]	255[FF]	251[FB]	2,128[0850]	8[08]	80[50]	0	0	0	0
18	NOOP	9800	0[0000]	0[00]	0[00]	47[002F]	0[00]	47[2F]	65,531[FFFB]	255[FF]	251[FB]	2,128[0850]	8[08]	80[50]	0	0	0	0
19	LIR AX, 2	3802	2[0002]	0[00]	2[02]	47[002F]	0[00]	47[2F]	65,531[FFFB]	255[FF]	251[FB]	2,128[0850]	8[08]	80[50]	0	0	0	0
20	LIR BX, 0	3880	2[0002]	0[00]	2[02]	0[0000]	0[00]	0[00]	65,531[FFFB]	255[FF]	251[FB]	2,128[0850]	8[08]	80[50]	0	0	0	0
21	INC BX	1880	2[0002]	0[00]	2[02]	1[0001]	0[00]	1[01]	65,531[FFFB]	255[FF]	251[FB]	2,128[0850]	8[08]	80[50]	0	0	0	0
22	CMP AX, BX	2810	2[0002]	0[00]	2[02]	1[0001]	0[00]	1[01]	65,531[FFFB]	255[FF]	251[FB]	2,128[0850]	8[08]	80[50]	0	0	1	0
23	JNZ, 21	9015	2[0002]	0[00]	2[02]	1[0001]	0[00]	1[01]	65,531[FFFB]	255[FF]	251[FB]	2,128[0850]	8[08]	80[50]	0	0	1	0
	INC BX	1880	2[0002]	0[00]	2[02]	2[0002]	0[00]	2[02]	65,531[FFFB]	255[FF]	251[FB]	2,128[0850]	8[08]	80[50]	0	0	0	0
	CMP AX, BX	2810	2[0002]	0[00]	2[02]	2[0002]	0[00]	2[02]	65,531[FFFB]	255[FF]	251[FB]	2,128[0850]	8[08]	80[50]	0	0	1	1
	JNZ, 21	9015	2[0002]	0[00]	2[02]	2[0002]	0[00]	2[02]	65,531[FFFB]	255[FF]	251[FB]	2,128[0850]	8[08]	80[50]	0	0	1	1
24	LIR AH, 3	3C03	770[0302]	3[03]	2[02]	2[0002]	0[00]	2[02]	65,531[FFFB]	255[FF]	251[FB]	2,128[0850]	8[08]	80[50]	0	0	1	1
25	LIR AL, 4	3C84	772[0304]	3[03]	4[04]	2[0002]	0[00]	2[02]	65,531[FFFB]	255[FF]	251[FB]	2,128[0850]	8[08]	80[50]	0	0	1	1
26	DEC AL	2480	771[0303]	3[03]	3[03]	2[0002]	0[00]	2[02]	65,531[FFFB]	255[FF]	251[FB]	2,128[0850]	8[08]	80[50]	0	0	0	0
27	CMP AH, AL	2C10	771[0303]	3[03]	3[03]	2[0002]	0[00]	2[02]	65,531[FFFB]	255[FF]	251[FB]	2,128[0850]	8[08]	80[50]	0	0	0	1
28	JZ, 26	881A	771[0303]	3[03]	3[03]	2[0002]	0[00]	2[02]	65,531[FFFB]	255[FF]	251[FB]	2,128[0850]	8[08]	80[50]	0	0	0	1
	DEC AL	2480	770[0302]	3[03]	2[02]	2[0002]	0[00]	2[02]	65,531[FFFB]	255[FF]	251[FB]	2,128[0850]	8[08]	80[50]	0	0	0	0
	CMP AH, AL	2C10	770[0302]	3[03]	2[02]	2[0002]	0[00]	2[02]	65,531[FFFB]	255[FF]	251[FB]	2,128[0850]	8[08]	80[50]	0	0	0	0
	JZ, 26	881A	770[0302]	3[03]	2[02]	2[0002]	0[00]	2[02]	65,531[FFFB]	255[FF]	251[FB]	2,128[0850]	8[08]	80[50]	0	0	0	0
29	ADD AX, [0]	0001	561[0231]	2[02]	49[31]	2[0002]	0[00]	2[02]	65,531[FFFB]	255[FF]	251[FB]	2,128[0850]	8[08]	80[50]	0	0	1	0
30	MOV AX, BX	3010	2[0002]	0[00]	2[02]	2[0002]	0[00]	2[02]	65,531[FFFB]	255[FF]	251[FB]	2,128[0850]	8[08]	80[50]	0	0	1	0
31	JMP 0	8000	2[0002]	0[00]	2[02]	2[0002]	0[00]	2[02]	65,531[FFFB]	255[FF]	251[FB]	2,128[0850]	8[08]	80[50]	0	0	1	0

Fig. 2. Trasarea programului încărcat în memoria RAM

6.2.2 Secvența de Execuție și Rezultate

Programul de test a urmat o secvență logică de operații, validând progresiv funcționalitățile cheie:

1. **Inițializare și Transfer Imediat:**
   - *Instrucțiuni:* LIR AX, 49 (0x0031), LIR BL, 12 (0x000C), LIR CX, 4.
   - *Validare:* S-a verificat afișarea corectă a valorilor 0031, 000C și 0004 pe display-ul cu 7 segmente la selectarea registrelor respective. Aceasta a confirmat funcționarea corectă a stărilor FETCH, DECODE și scrierea în RegFile.
2. **Aritmetică și Flag-uri:**
   - *Instrucțiuni:* ADD DX, CX, SUB AX, CX, INC DX.
   - *Validare:* S-a verificat rezultatul numeric în registre și, critic, aprinderea LED-urilor corespunzătoare flag-urilor. De exemplu, o scădere care a rezultat într-o valoare negativă a activat LED-ul pentru *Sign Flag* (SF), validând logica **ALU**.
3. **Accesul la Memorie (Load/Store):**
   - *Instrucțiuni:* STD AX, 0 (Store la adresa 0), LDR BX, 0 (Load de la adresa 0).
   - *Validare:* Această secvență a testat magistrala de date bidirecțională. S-a verificat că valoarea salvată din AX a fost recuperată corect în BX, confirmând funcționarea corectă a logicii de calcul a adresei fizice în **BIU**.
4. **Controlul Fluxului (Salturi):**
   - *Instrucțiuni:* CMP AX, BX urmat de JNZ (Jump if Not Zero) sau JZ.
   - *Validare:* Acesta a fost testul suprem pentru decuplarea **BIU-EU**. La întâlnirea instrucțiunii de salt condiționat, s-a confirmat că instrucțiunile aflate deja în coada FIFO (vizibile în format hexazecimal pe display-ul de debug) au fost ignorate (golite). Acest lucru a validat prioritatea semnalului FLUSH și capacitatea procesorului de a modifica

fluxul de execuție, concomitent cu avansarea Contorului de Instrucțiuni (IP) către noua adresă țintă.

6.3 Analiza Rezultatelor

Execuția hardware pas-cu-pas a confirmat corespondența perfectă între comportamentul procesorului implementat și tabelul de referință din Excel. Toate instrucțiunile testate s-au comportat conform specificațiilor arhitecturale 8086:

- Registrele au fost actualizate corect, respectând distincția dintre accesul pe 8 și 16 biți.
- Unitatea de Interfață cu Magistrala (BIU) a menținut coada de instrucțiuni plină în momentele de execuție liniară și a răspuns corect la cererile de salt.
- Flag-urile au reflectat fidel rezultatul operațiilor, permițând executarea corectă a buclelor și deciziilor logice.

Validarea a demonstrat robustețea designului multi-ciclu și capacitatea acestuia de a executa programe complexe într-un mediu real FPGA.

7. Concluzii

Proiectul de implementare a unității centrale de procesare Intel 8086 într-o arhitectură simplificată multi-ciclu pe platforma FPGA a îndeplinit cu succes obiectivele stabilite inițial.

7.1 Obiective Atinse și Contribuții

Designul a reușit să replice modelul de programare 8086, menținând în același timp un nivel ridicat de fezabilitate VHDL prin adoptarea unor decizii arhitecturale cheie:

- **Arhitectura BIU/EU:** S-a realizat o separare funcțională clară între Unitatea de Interfață cu Magistrala (BIU) și Unitatea de Execuție (EU), emulând principiul de pre-fetch al arhitecturii originale.
- **Adresarea Segmentată:** S-a implementat cu succes logica de calcul a adresei fizice de 20 de biți (Segment x 16 + Offset).
- **Decuplarea Hazarde/Simplificare:** Prin adoptarea unei unități de control bazate pe stări (Multi-Cycle) și prin standardizarea instrucțiunilor la o lungime fixă de 16 biți, s-a eliminat complexitatea gestionării hazardelor de pipeline, asigurând o secvențiere robustă a instrucțiunilor.

7.2 Validare și Rezultate

Validarea hardware directă, realizată cu ajutorul Generatorului de Puls Unic (MPG) și a interfeței de afișare (SSD), a confirmat funcționalitatea integrală a Datapath-ului. Testele au demonstrat că sistemul:

- Execută corect operațiile aritmetice și logice, cu o actualizare fidelă a Flag-urilor (ZF, CF, SF, OF).
- Menține integritatea logică la accesul dual (8/16 biți).

7.3 Perspective de Viitor

Proiectul curent servește ca o bază solidă pentru extinderi viitoare. Designul modular permite adăugarea relativ facilă a următoarelor elemente:

- **Suport pentru Instrucțiuni Variabile (CISC):** Cea mai importantă extindere presupune modificarea Unității de Control (CU) și a BIU pentru a gestiona instrucțiunile de lungime variabilă (1-6 octeți), renunțând la simplificarea lungimii fixe de 16 biți. Aceasta ar necesita implementarea unei logici complexe de pre-fetch și decodificare secvențială, conform standardului x86 original.
- **Utilizarea Integrală a Segmentelor:** Activarea completă a registrelor de segment SS (Stack Segment) și ES (Extra Segment) prin implementarea instrucțiunilor PUSH/POP și a operațiilor pe șiruri de caractere (string operations) care folosesc explicit adresele SS:SP și ES:DI.
- **Set de Instrucțiuni Extins:** Implementarea operațiilor I/O și a instrucțiunilor de manipulare a stivei (PUSH, POP), utilizând registrele de pointeri deja definite (SP, BP).
- **Suport pentru Întreruperi:** Adăugarea logicii hardware pentru a gestiona întreruperile (hardware/software), sporind complexitatea Unității de Control.

În concluzie, prin aplicarea principiilor de proiectare digitală *bottom-up* și a limbajului VHDL, s-a demonstrat că o arhitectură istorică, complexă, poate fi redusă și implementată funcțional într-un mediu FPGA, oferind o înțelegere profundă a structurii sistemelor de calcul.

8. Bibliografie

[1] https://www.tutorialspoint.com/microprocessor/microprocessor_8086_overview.htm

[2] https://www.geeksforgeeks.org/computer-organization-architecture/architecture-of-8086

[3] https://www.scribd.com/document/690546349/INTEL-8086-Microprocessor-Architecture?v=0.523

[4] https://www.lkouniv.ac.in/site/writereaddata/siteContent/202004050627538832priyanka_engg_Microprocessor.pdf

[5] https://www.skdavpolytech.ac.in/news_files/microprocessor_part_3_compressed_1588259301.pdf

[6] https://electronicsdesk.com/8086-microprocessor.html

[7] https://www.secs.oakland.edu/~llamocca/Courses/ECE2700/F17/FinalProject/Group1_8086.pdf

[8] https://ia903207.us.archive.org/15/items/Programming_the_8086_8088/Programming_the_8086_8088.pdf

[9] https://vardhaman.org/wp-content/uploads/2021/03/Unit-1-MPMC.pdf

[10] https://userpages.umbc.edu/~squire/intel_book.pdf

[11] https://mlgansari.wordpress.com/wp-content/uploads/2020/04/3-instruction-set-of-8086-microprocessor.pdf

[12] Gemini, ChatGPT